ZERO-KNOWLEDGE PROOF-BASED AUTHENTICATION FRAMEWORK FOR CLOUD COMPUTING

A

THESIS

Submitted in partial fulfillment of the requirements for the award of degree of

MASTERS OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE

Submitted by

अभय दांडगे

ABHAY DANDGE

Scholar Number: **24215011117**

Under the Guidance of

डॉ. नमिता तिवारी

Dr. Namita Tiwari

Assistant Professor, Department of CSE, MANIT Bhopal

डॉ. मीनू चावला

Dr. Meenu Chawla

Professor, Department of CSE, MANIT Bhopal

कृत्रिम बुद्धिमत्ता केंद्र

Centre Of Artificial Intelligence

मौलाना आज़ाद राष्ट्रीय प्रौद्योगिकी संस्थान भोपाल

MAULANA AZAD NATIONAL INSTITUTE OF TECHNOLOGY BHOPAL

JULY 2026

मौलाना आज़ाद राष्ट्रीय प्रौद्योगिकी संस्थान भोपाल

MAULANA AZAD NATIONAL INSTITUTE OF TECHNOLOGY BHOPAL

(An Institute of National Importance)

Bhopal - 462003

कृत्रिम बुद्धिमत्ता केंद्र

Centre Of Artificial Intelligence

Declaration

I hereby declare that the work presented in the dissertation entitled **"Zero-Knowledge Proof-Based Authentication Framework for Cloud Computing"** is submitted in partial fulfillment of the requirements for the award of the degree of **Masters of Technology** in **Artificial Intelligence**. It is an authentic documentation of my own original work carried out from July 2024 to July 2026 under the guidance of **Dr. Namita Tiwari**, Assistant Professor, and **Dr. Meenu Chawla**, Professor, Department of CSE, MANIT Bhopal.

Wherever contributions of others are involved, every effort is made to indicate this clearly with due reference to the literature. The matter embodied in this dissertation has not been presented or submitted for any other degree.

Abhay Dandge
Scholar Number: 24215011117
M.Tech (Artificial Intelligence)
MANIT, Bhopal

मौलाना आज़ाद राष्ट्रीय प्रौद्योगिकी संस्थान भोपाल

MAULANA AZAD NATIONAL INSTITUTE OF TECHNOLOGY BHOPAL

(An Institute of National Importance)

Bhopal - 462003

कृत्रिम बुद्धिमत्ता केंद्र

Centre Of Artificial Intelligence

Certificate

This is to certify that the dissertation entitled **"Zero-Knowledge Proof-Based Authentication Framework for Cloud Computing"** submitted by **Abhay Dandge**, Scholar No. 24215011117, in partial fulfillment of the requirements for the degree of **Masters of Technology** in Artificial Intelligence, is an authentic work carried out under our supervision and guidance from July 2024 to July 2026.

Dr. Namita Tiwari	**Dr. Meenu Chawla**
Supervisor	Supervisor
Assistant Professor, Dept. of CSE	Professor, Dept. of CSE
MANIT, Bhopal	MANIT, Bhopal

Acknowledgement

I would like to begin by expressing my profound sense of gratitude towards my supervisors **Dr. Namita Tiwari**, Assistant Professor, and **Dr. Meenu Chawla**, Professor, Department of CSE, MANIT Bhopal, for their valuable guidance, support, and encouragement throughout this research. Their readiness for consultation at all times, their educative comments, and their constant concern for quality have been invaluable.

I express my sincere gratitude to our Director, **Dr. K.K. Shukla**, for providing ample resources and a conducive research environment. I extend my thanks to **Dr. R.K. Pateriya**, Head of Department of CSE, and **Dr. D.S. Tomar**, Head of the Centre Of Artificial Intelligence, for administrative support and for providing access to departmental laboratories.

I am grateful to my co-author **Sameeksha Prasad** for collaborative research discussions that contributed to the published works underpinning this thesis. I also thank my fellow research scholars and classmates for stimulating academic discussions and collegial support throughout the programme.

Finally, I am deeply indebted to my parents and family for everything they have given me. They have always stood by me with constant support, encouragement, and love through every phase of this research.

Date:

Abhay Dandge
Scholar Number: 24215011117

Place: Bhopal

Abstract

Authentication security in cloud environments remains a critical vulnerability. Traditional mechanisms—passwords, multi-factor authentication (MFA), and public key infrastructure (PKI)—rely on a *verify-by-reveal* paradigm. Sensitive credentials must be transmitted or stored by the verifier, creating massive attack surfaces highly exploitable through database breaches and credential stuffing.

This thesis introduces ZK-Auth, a cryptographic infrastructure resolving the verify-by-reveal problem. ZK-Auth operates as an OAuth 2.0 Authorization Server where the standard password authentication step is replaced by a zero-knowledge proof. Three core mechanisms realize this design. *First*, users prove knowledge of a registration secret strictly on the client side using Groth16 zk-SNARKs over the BN254 elliptic curve, compiled to WebAssembly. The resulting proof authorizes a short-lived OAuth authorization code. Poseidon hash-based nullifiers simultaneously prevent replay attacks.

Second, selective credential disclosure is achieved via depth-8 Poseidon Merkle trees integrated with W3C Verifiable Credentials. Holders prove individual attribute predicates (e.g., age ≥ 18) without exposing raw attribute values. *Third*, ZK-Auth incorporates a two-layer LSTM behavioral biometric model providing background session-risk scoring derived from user telemetry, triggering step-up re-authentication upon detecting anomalous behavior.

The architecture supports multi-tenant institutional issuance via Ed25519 signing keypairs. ZK-Auth is implemented end-to-end and evaluated across Node.js, Chrome, and two Android devices. It achieves a median end-to-end authentication latency of 132.5 ms—23% faster than an Argon2id password-hashing baseline—with a 723-byte proof. The behavioral biometric subsystem, evaluated on the public Balabit Mouse Dynamics dataset, achieves an AUC-ROC of 0.816 with a 6.15% false-acceptance rate. This zero-PII architecture structurally complies with India's DPDP Act 2023 and the GDPR.

संक्षेप

क्लाउड परिवेशों में प्रमाणीकरण सुरक्षा अब भी एक गंभीर भेद्यता बनी हुई है। पारंपरिक तंत्र—पासवर्ड, बहु-कारक प्रमाणीकरण एमएफए, और सार्वजनिक-कुंजी अवसंरचना पीकेआई—एक प्रकट करके सत्यापित करें प्रतिमान पर निर्भर करते हैं। संवेदनशील क्रेडेंशियल्स को सत्यापनकर्ता द्वारा प्रेषित या संग्रहीत करना आवश्यक होता है, जिससे विशाल आक्रमण सतहें बनती हैं, जिनका डेटाबेस उल्लंघनों और क्रेडेंशियल स्टफिंग के माध्यम से अत्यधिक शोषण किया जा सकता है।

यह शोधप्रबंध जेडके-ऑथ प्रस्तुत करता है, जो एक ऐसी क्रिप्टोग्राफिक अवसंरचना है जो परकट करके सत्यापित करें की समस्या का समाधान करती है। जेडके-ऑथ एक $\mathrm{OAuth}\ 2.0$ प्राधिकरण सर्वर के रूप में कार्य करता है, जहाँ मानक पासवर्ड प्रमाणीकरण चरण को शून्य-ज्ञान प्रमाण (शून्य-ज्ञान प्रमाण) द्वारा प्रतिस्थापित किया जाता है। इस डिज़ाइन को तीन मुख्य तंत्र साकार करते हैं। पहला, उपयोगकर्ता ग्रोथ16 $\mathrm{zk\text{-}SNARKs}$ का उपयोग करते हुए, बीएन254 एलिप्टिक वक्र पर, वेबअसेंबली में संकलित प्रणाली के माध्यम से, केवल क्लाइंट-पक्ष पर पंजीकरण रहस्य (पंजीकरण गुप्त) के ज्ञान का प्रमाण प्रस्तुत करते हैं। परिणामी प्रमाण एक अल्पकालिक ओएथ प्राधिकरण कोड को अधिकृत करता है। $\mathrm{Poseidon}$ हैश-आधारित नलिफ़ायर्स (निरस्त करने वाले) एक साथ रिप्ले आक्रमणों को रोकते हैं।

दूसरा, चयनात्मक क्रेडेंशियल प्रकटीकरण (चयनात्मक क्रेडेंशियल प्रकटीकरण) गहराई-8 पोसाइडन मर्कले वृक्षों के माध्यम से प्राप्त किया जाता है, जिन्हें $\mathrm{W3C}$ सत्यापन योग्य क्रेडेंशियल के साथ एकीकृत किया गया है। धारक कच्चे गुण मानों (कच्ची विशेषता मान) को उजागर किए बिना व्यक्तिगत गुण-आधारित विधेयों (जैसे, आयु ≥ 18) का प्रमाण प्रस्तुत करते हैं। तीसरा, जेडके-ऑथ एक द्वि-स्तरीय एलएसटीएम व्यवहारिक बायोमेट्रिक मॉडल को सम्मिलित करता है, जो उपयोगकर्ता टेलीमेट्री से प्राप्त पृष्ठभूमि सत्र-जोखिम स्कोरिंग प्रदान करता है और असामान्य व्यवहार का पता चलने पर उन्नत पुनः-प्रमाणीकरण (स्टेप-अप पुनः प्रमाणीकरण) को सक्रिय करता है।

यह वास्तुकला $\mathrm{Ed25519}$ हस्ताक्षर कुंजी-युग्मों (कुंजी युग्मों पर हस्ताक्षर करना) के माध्यम से बहु-किरायेदार (बहु-किरायेदार) संस्थागत निर्गमन का समर्थन करती है। जेडके-ऑथ को पूर्णतः कार्यान्वित किया गया है और नोड.जेएस, करोम तथा दो एंड्रॉयड उपकरणों पर मूल्यांकित किया गया है। यह 132.5 मिलीसेकंड की माध्य अंत-से-अंत प्रमाणीकरण विलंबता प्राप्त करता है—जो आर्गन2आईडी पासवर्ड-हैशिंग आधाररेखा की तुलना में 23% तेज़ है—और इसका प्रमाण आकार 723 बाइट है। व्यवहारिक बायोमेट्रिक उपप्रणाली, जिसे सार्वजनिक बालाबिट माउस डायनेमिक्स डेटासेट पर मूल्यांकित किया गया, 0.816 का एयूसी-आरओसी तथा 6.15% की गलत-स्वीकृति दर (झूठी स्वीकृति दर) प्राप्त करती है। यह शून्य-पीआईआई वास्तुकला भारत के डीपीडीपी अधिनियम 2023 तथा जीडीपीआर के साथ संरचनात्मक रूप से अनुपालन करती है।

Contents

List of Tables

List of Figures

List of Abbreviations

Abbreviation	Full Form
ABE	Attribute-Based Encryption
API	Application Programming Interface
AUC	Area Under the (ROC) Curve
BN254	Barreto-Naehrig 254-bit elliptic curve
CA	Certificate Authority
CP-ABE	Ciphertext-Policy Attribute-Based Encryption
CRS	Common Reference String
DID	Decentralized Identifier
DPDP	Digital Personal Data Protection (Act, India 2023)
EMA	Exponential Moving Average
EUF-CMA	Existential Unforgeability under Chosen Message Attack
FAR	False Acceptance Rate
FRR	False Rejection Rate
GDPR	General Data Protection Regulation
gRPC	Google Remote Procedure Call
IAM	Identity and Access Management
IdP	Identity Provider
IoT	Internet of Things
JWT	JSON Web Token
KZG	Kate-Zaverucha-Goldberg polynomial commitment
LSTM	Long Short-Term Memory
MFA	Multi-Factor Authentication
NIZK	Non-Interactive Zero-Knowledge
OAuth	Open Authorization
PKI	Public Key Infrastructure
PII	Personally Identifiable Information
PPT	Probabilistic Polynomial Time
QAP	Quadratic Arithmetic Program
R1CS	Rank-1 Constraint System
SAML	Security Assertion Markup Language
SISMEMBER	Redis Set-Is-Member command

Abbreviation	Full Form
SNARK	Succinct Non-interactive ARgument of Knowledge
STARK	Scalable Transparent ARgument of Knowledge
SSI	Self-Sovereign Identity
TEE	Trusted Execution Environment
TTL	Time To Live
VC	Verifiable Credential
VP	Verifiable Presentation
W3C	World Wide Web Consortium
WASM	WebAssembly
WebView	Embedded browser engine within a native mobile app
ZKP	Zero-Knowledge Proof
ZK-SNARK	Zero-Knowledge Succinct Non-Interactive Argument of Knowledge
ZK-STARK	Zero-Knowledge Scalable Transparent Argument of Knowledge

Chapter 1

INTRODUCTION

1.1 Overview

This chapter introduces the problem this thesis addresses, the design philosophy behind ZK-Auth, and the structure of the remaining chapters. The discussion begins with why password-centric authentication is becoming difficult to defend at cloud scale, moves through the specific gaps in existing zero-knowledge authentication proposals, and ends with a summary of what this thesis contributes and how it is organised.

1.2 Background

Cloud computing has transformed how digital services are delivered, with the global cloud market exceeding $600 billion annually by 2024. Alongside this growth has come a harder security problem: how does a service authenticate a user when the infrastructure handling that authentication is not, in the traditional sense, under the service provider's direct physical control?

Most authentication systems in use today—passwords, multi-factor authentication (MFA), and public key infrastructure (PKI)—were designed for an earlier assumption: that the server checking a credential could be fully trusted with that credential. In a cloud deployment this assumption is weaker than it used to be. The result is what this thesis refers to as the *verify-by-reveal* paradigm: to prove who you are, you hand over (or allow the server to store) something secret, and the server's job is to check it against a stored copy. Every large-scale credential breach in the last decade—from password database leaks to credential-stuffing campaigns that replay leaked passwords across unrelated services—traces back to this same underlying design choice.

Zero-Knowledge Proofs (ZKPs) offer a different answer. A ZKP lets a *prover* convince a *verifier* that a statement is true without revealing anything else about why it is true [2]. Applied to login, this means a user can prove they know a secret without ever sending that secret anywhere. If the server's database is breached, there is no secret in it to steal.

1.3 Topic Introduction

ZK-Auth, the system built and evaluated in this thesis, is best understood as three smaller problems solved together rather than one big one. Each is introduced briefly here and developed in full in later chapters.

1.3.1 Zero-Knowledge Authentication

The first problem is replacing the password check itself. ZK-Auth uses Groth16 zk-SNARKs over the BN254 elliptic curve, compiled from a Circom circuit to WebAssembly so the proof can be generated entirely on the user's device. The server never sees the secret, only a proof that the user knows it. This is described fully in Chapter 3.

1.3.2 Selective Attribute Disclosure

The second problem is that real applications rarely need to know everything about a user—a liquor delivery app needs to know that a customer is over 18, not their exact birth date. ZK-Auth addresses this with depth-8 Poseidon Merkle trees, letting a credential holder prove a predicate about one attribute (age, nationality, membership tier) without revealing the attribute's value or any other attribute stored alongside it. This is covered in Chapter 4.

1.3.3 Continuous Session Assurance

The third problem is that authenticating once at login time says nothing about whether the same person is still in control of the session five minutes later. ZK-

AUTH layers a behavioral biometric model—an LSTM network watching mouse and keyboard telemetry—on top of the ZKP login, and forces a fresh proof whenever the model's risk score crosses a threshold. This is also covered in Chapter 4.

1.4 Motivation

Despite the theoretical appeal of zero-knowledge authentication, very few production systems use it, and the published research on the topic has left several gaps that motivated this work directly.

First, almost every prior ZKP-authentication paper benchmarks proof generation on a single machine—usually the author's own development laptop. This says little about whether the approach is workable for a real user base logging in from a mix of laptops, phones, and tablets. Second, plain ZKP authentication on its own does not let a verifier ask for anything less than full proof of identity; there is no native way to ask only "are you over 18" without exposing more. Third, a successful login proof is a point-in-time guarantee, and most ZKP-authentication work stops there, leaving session hijacking after login as an open problem. Fourth, much of the literature presents security arguments at the level of "Groth16 is secure, therefore our system is secure," without working through how the specific circuit used actually inherits that guarantee.

A fifth, more practical concern also shaped this thesis: an institution that already has OAuth-based login integrations with its partners cannot realistically rip that out to adopt a new authentication scheme. Most prior ZKP-authentication proposals implicitly assume they will replace the whole login stack, including the relying-party delegation layer that OAuth already provides. ZK-AUTH takes a different position: it is built *as* an OAuth 2.0 Authorization Server, and the only thing it changes is what happens during the password-entry step of OAuth's Authorization Code grant — that step becomes a Groth16 proof instead of a password form. This is the same pattern used by "Sign in with Apple," which pairs OAuth/OIDC delegation with Face ID or Touch ID rather than a password. Section 1.6 lists how this thesis addresses each of these five gaps directly.

Table 1.1 summarises, at a glance, how the verify-by-reveal paradigm compares against the zero-knowledge approach taken in this thesis; Chapter 6 expands this into a full seven-system comparison with measured and literature-reported fig-

ures.

Table 1.1: Verify-by-Reveal vs. Zero-Knowledge Authentication

Property	**Verify-by-Reveal**	**Zero-Knowledge (ZK-Auth)**
Secret leaves client	Yes (password/hash)	Never
Database breach impact	Credentials exposed	Only commitments exposed
Replay resistance	Requires extra design	Built into protocol (nullifiers)
Attribute-level privacy	Not native	Native (Merkle predicates)
Works within OAuth 2.0	Yes	Yes (this thesis)

Figure 1.1 illustrates the structural difference between the two paradigms at the point where a credential would normally be transmitted.

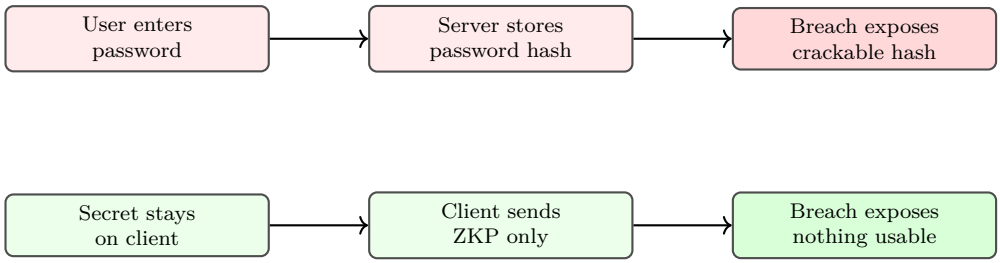

Figure 1.1: Verify-by-reveal (top) vs. zero-knowledge authentication (bottom).

1.5 Significance of the Study

This work matters beyond its immediate technical contributions for three concrete reasons. First, the regulatory landscape for personal data is tightening: India's Digital Personal Data Protection Act 2023 and the EU's GDPR both penalize unnecessary collection and storage of personal data, and a system that authenticates users without ever storing a usable secret is, by construction, easier to bring into compliance than one built on conventional password storage. Second, credential-theft remains one of the most common root causes of large-scale breaches year after year; an authentication primitive that removes the stored secret entirely removes that root cause rather than mitigating its symptoms. Third, most prior academic proposals in this space stop at a working prototype evaluated on a single machine; this thesis instead treats cross-environment deployability, ablation-justified design choices, and a formal security reduction as first-class deliverables, so that the result is closer to something an engineering team could actually adopt than a proof-of-concept confined to a research paper.

The practical audience for this work includes cloud service providers evaluating alternatives to password-based login, identity-platform teams exploring privacy-preserving authentication, and researchers working on the intersection of zero-knowledge cryptography and applied systems security. The OAuth-compatibility design choice (Section 3.6) specifically targets the first group: an institution does not need to discard its existing relying-party integrations to adopt the approach described here.

1.6 Research Contributions

This thesis presents ZK-Auth, a complete cryptographic authentication infrastructure addressing the gaps identified above in an integrated, empirically evaluated, and formally analysed system. The specific contributions are:

1. **ZK-Auth System:** A complete implementation of Groth16 zk-SNARK authentication over BN254 using Circom-compiled circuits, deployed as browser-executable WebAssembly with Node.js server-side verification, wrapped in a working OAuth 2.0 Authorization Code flow with PKCE so the proof substitutes only for the password-entry step rather than for OAuth's relying-party delegation layer.

2. **Poseidon Merkle Selective Disclosure:** A depth-8 Poseidon Merkle tree commitment scheme for W3C VC selective attribute disclosure supporting GTE, LTE, and EQ predicate proofs without attribute value exposure, with an ablation analysis (Chapter 5) justifying the depth-8 design point.

3. **LSTM Behavioral Biometric Continuity:** A two-layer LSTM model on a sliding window of six telemetry features, with EMA smoothing and automated ZKP step-up re-authentication, trained and evaluated on the public Balabit Mouse Dynamics Challenge dataset rather than synthetic data, with a window-size ablation (Chapter 5) justifying the 50-event design point.

4. **Multi-Tenant Institutional Issuance:** A PKI-analogous credential issuance platform with per-institution Ed25519 keypair generation, W3C DID publication, and institutional credential sovereignty.

5. **Poseidon Dart Implementation:** A BN254 Poseidon library in Dart with circomlib-compatible constants, enabling mobile Poseidon-correct commitment computation without external dependencies.

6. **Cross-Environment Empirical Validation:** A four-environment benchmark suite (Node.js server, Chrome browser via Web Worker, and two physical Android devices of differing hardware tier) demonstrating that proof generation remains within sub-second latency across the full measured hardware spread, together with a formal extraction-based soundness reduction tying the specific Poseidon-bound circuit constraints to Groth16's generic-group guarantees.

7. **Baseline Comparison:** A seven-paradigm comparison (Password, MFA, PKI, pure OAuth, SAML, WebAuthn, and behavioral-biometric-only systems) combining qualitative feature analysis with literature-reported quantitative figures (Chapter 6).

1.7 Scope and Delimitations

1. Groth16 over BN254 is used as the proof system throughout the implemented prototype; STARKs, PLONK, and Bulletproofs are analyzed comparatively in Chapter 2 but not implemented as alternative backends in the primary system.

2. Browser (Chrome/Web Worker) and native mobile (Flutter/WebView) clients are the proof-generation environments empirically evaluated; native iOS WKWebView benchmarking is identified as future work (Chapter 7) due to device unavailability during the evaluation period.

3. The behavioral biometric model is trained and evaluated on the public Balabit Mouse Dynamics Challenge dataset, which provides mouse-only telemetry; multi-modal (keyboard, scroll) extension is identified as future work, since the live telemetry pipeline collects six features but the public training corpus exercises only the mouse-derived subset.

4. The public Hermez Network Powers of Tau ceremony is used for the Groth16 trusted setup; a deployment-specific ceremony is discussed as a hardening option in Chapter 7 but not executed in this thesis.

5. Multi-institution concurrent-load evaluation of the issuance layer (Chapter 4) is functionally validated but not load-tested under simultaneous multi-tenant traffic; this is scoped as future work.

1.8 Organization of Thesis

Chapter 2 surveys the literature underpinning this thesis and develops the theoretical background necessary for the constructions in later chapters, spanning zero-knowledge proof systems, the Groth16 construction, ZKP-cloud authentication architectures, attribute-based encryption integration, selective disclosure standards, and behavioral biometrics for continuous authentication. Chapter 3 develops the core proposed system architecture, threat model, and ZKP authentication protocol — including the precise relationship between OAuth 2.0 and the ZKP authentication primitive — with a full constraint-level treatment of the authentication circuit. Chapter 4 presents the advanced subsystems: selective disclosure, the behavioral biometric subsystem, and multi-tenant institutional issuance. Chapter 5 details the implementation and presents the complete empirical evaluation, including ablation studies, across all four measured deployment environments. Chapter 6 provides a full baseline comparison against seven existing authentication paradigms with literature-reported quantitative figures. Chapter 7 provides an extended security analysis with formal proofs and discusses limitations. Chapter 8 outlines future work and concludes the thesis.

Chapter 2

LITERATURE SURVEY AND THEORETICAL BACKGROUND

This chapter synthesizes the literature underpinning this thesis, drawing on both published review papers [12, 22], and details the theoretical foundations required for the constructions presented in Chapters 3 and 4.

2.1 Zero-Knowledge Proof Systems

Zero-Knowledge Proofs were introduced by Goldwasser, Micali, and Rackoff [1], establishing completeness, soundness, and zero-knowledge as the three foundational properties a proof system must satisfy. Completeness requires that an honest prover convinces an honest verifier with overwhelming probability; soundness requires that no cheating prover can convince the verifier of a false statement except with negligible probability; zero-knowledge requires that the verifier's view of an interaction can be simulated without access to the prover's secret witness. Blum, Feldman, and Micali [3] introduced non-interactive ZKPs via a common reference string (CRS), removing the requirement for multi-round interaction between prover and verifier—a prerequisite for the single-round authentication protocol developed in Chapter 3. The Fiat–Shamir heuristic [4] transformed interactive protocols into non-interactive ones using hash functions as random oracles, a technique foundational to the construction of practical NIZK and SNARK systems including Groth16.

Practical succinct non-interactive arguments of knowledge (SNARKs) became computationally feasible with Pinocchio [5] and the quadratic arithmetic program (QAP) formulation of Gennaro et al. [6], which reduces an arbitrary boolean or arithmetic circuit to a polynomial identity checkable via a constant number of group operations. Groth16 [7] achieved constant-size proofs (three group elements) with $\mathcal{O}(1)$ pairing-based verification—the most efficient known SNARK construction under the generic group model for a fixed circuit, and the construction used through-

out this thesis. Groth16 has been deployed in Zcash [8] for shielded transactions, demonstrating practical viability at scale, albeit with proof-generation times on the order of seconds for cryptocurrency-scale circuits substantially larger than the authentication circuit developed in this thesis. PLONK [9] offers a universal (circuit-independent) trusted setup at the cost of larger proofs and higher verification overhead; STARKs [10] offer a fully transparent setup, eliminating the trusted-setup ceremony entirely, but incur proof sizes an order of magnitude larger than Groth16, which is prohibitive for the latency- and bandwidth-sensitive authentication context targeted here. Bulletproofs [11] avoid trusted setup for range proofs specifically but scale logarithmically rather than constantly with statement complexity.

2.1.1 Groth16 zk-SNARK

Groth16 produces proofs $\pi = (\pi_A, \pi_B, \pi_C) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$. Verification checks:

$$e(\pi_A, \pi_B) = e(\alpha, \beta) \cdot e\left(\sum_i x_i \gamma_i, \gamma\right) \cdot e(\pi_C, \delta) \tag{2.1}$$

requiring only three pairing operations regardless of circuit size. All circuit signals are elements of $\mathbb{Z}_r$ over the BN254 elliptic curve, which provides approximately 110-bit security against known attacks on the embedding degree, and is the curve used by the `snarkjs/circomlib` toolchain employed in this thesis's implementation.

The Groth16 setup phase generates a structured reference string (SRS) specific to a given circuit's QAP representation. This SRS is derived from a universal, circuit-independent "Powers of Tau" ceremony followed by a circuit-specific second phase. The security of the resulting proof system rests on the assumption that at least one participant in the original ceremony destroyed their secret randomness (the so-called "toxic waste"); this is formalized as the q-Power Knowledge of Exponent (q-PKE) assumption in the generic group model, which underlies the soundness proof developed formally in Chapter 7.

2.2 ZKP for Cloud Authentication

Our prior review [12] surveyed 28 papers (2019–2025), establishing two primary deployment architectures for ZKP-based cloud authentication.

Stand-alone ZKP: The cloud service itself acts as verifier, with no blockchain or distributed-ledger intermediary. Khandait and Pattanshetti [13] demonstrated lightweight ZKP authentication for IoT devices achieving low latency and high accuracy on resource-constrained hardware. Pathak et al. [14] combined ZKP with the Secure Remote Password protocol for anonymous authentication, showing that ZKP techniques compose well with existing password-derived protocols rather than requiring a complete authentication-stack rewrite. Butt et al. [15] demonstrated FPGA-based multi-scalar multiplication (MSM) acceleration, directly relevant to future hardware acceleration of the proving step that dominates client-side latency in Chapter 5's benchmarks.

Ledger-anchored ZKP: Smart contracts handle proof verification on-chain, trading verification cost and latency for public auditability. Hou et al. [16] implemented blockchain-ZKP energy trading. Verma et al. [17] implemented healthcare access control with on-chain ZKP verification. Naidu et al. [18] achieved 94.44% accuracy in ZKP-based payment authentication. Table 2.1 establishes that Groth16 stand-alone verification is optimal for high-traffic cloud APIs, which directly motivated the system-level design decision in Chapter 3 to use stand-alone (non-ledger-anchored) Groth16 verification: ZK-Auth targets per-request authentication latency in the tens of milliseconds (Chapter 5), a regime in which on-chain verification's block-confirmation delay (typically seconds) is incompatible with the system's latency goals.

Table 2.1: ZKP Family Comparison for Cloud Authentication.

ZKP Family	**Setup**	**Proof Size**	**Verif. Cost**	**Cloud Fit**
Σ-protocols	None	Small	Low	Stand-alone
zk-SNARKs (Groth16)	Trusted	$\approx$288B	$\mathcal{O}(1)$	Both
Univ. SNARKs	Universal	Small	$\mathcal{O}(1)$	Both
zk-STARKs	Transparent	>40KB	Low	Ledger
Bulletproofs	None	Medium	$\mathcal{O}(N)$	Stand-alone

The proof-size figure of $\approx$288 bytes in Table 2.1 reflects the theoretical minimum (three uncompressed BN254 group elements); the implemented system's measured serialized proof size of 723 bytes median (Chapter 5) reflects the JSON serialization overhead of the actual `snarkjs` proof object format, including field-element string encoding, and is reported transparently as the realistic deployment figure rather than the theoretical lower bound.

Figure 2.1 illustrates the two deployment architectures distinguished above

and the design point this thesis adopts. Stand-alone verification keeps the verifier as the sole trust anchor (lower latency, no consensus delay); ledger-anchored verification trades that latency for a publicly auditable verification record on a blockchain.

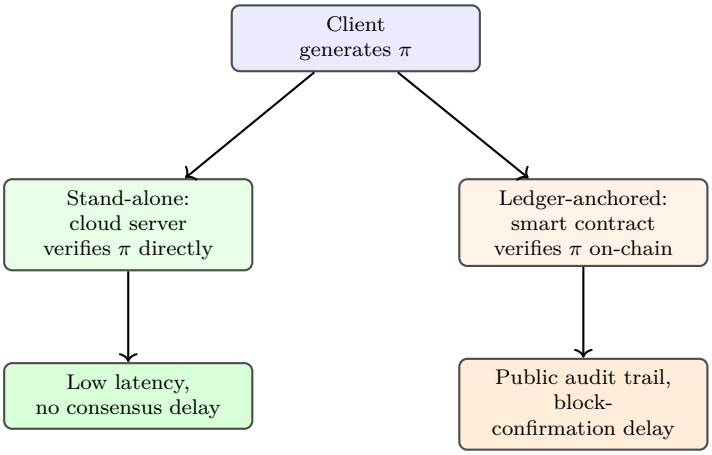

Figure 2.1: Stand-alone vs. ledger-anchored ZKP deployment architectures.

2.3 ZKP and Attribute-Based Encryption

Attribute-Based Encryption (ABE) was introduced by Sahai and Waters [19] and extended to ciphertext-policy ABE (CP-ABE) by Goyal et al. [20], enabling fine-grained access control where a ciphertext can only be decrypted by users whose attributes satisfy an embedded access policy. Traditional ABE decryption requires $\mathcal{O}(N)$ bilinear pairings for N attributes, creating latency that becomes impractical for large attribute sets—a scalability limitation directly analogous to the linear Merkle-path cost analyzed in the depth ablation of Chapter 5. The KZG polynomial commitment scheme [21] provides $\mathcal{O}(1)$ verification independent of polynomial degree, offering a template for how ZK-Auth's Merkle approach achieves $\mathcal{O}(\log n)$ (rather than $\mathcal{O}(n)$) disclosure proof cost relative to naive per-attribute ABE decryption. Our review [22] surveyed ZKP-ABE integration approaches including Yang et al. [23], who apply ZKP-based distributed attribute encryption to virtual power plant privacy, and Ren et al. [24], who combine blockchain-based CP-ABE data sharing with distributed key management and ZKP. These works motivate ZK-Auth's selective disclosure design (Chapter 4) as a lighter-weight alternative to full ABE for the specific case of boolean predicate disclosure over a bounded, individually-owned attribute set, rather than the general ciphertext-policy access control that ABE targets.

2.4 Selective Disclosure and W3C Standards

BBS+ signatures [25] support message-level disclosure, allowing a holder to reveal a subset of signed messages while keeping others hidden, but the revealed messages are still exposed to the verifier in cleartext form—satisfying data minimization in the weak sense (only requested attributes are shown) but not the strong sense (the requested attribute's literal value is still disclosed). SD-JWT [26] is a practical, widely-deployable mechanism following the same exposure model as BBS+ but without the zero-knowledge property. CL credentials [27] provide genuine zero-knowledge disclosure (the verifier learns only the predicate result, not the value) but were designed prior to the emergence of SNARK-friendly hash functions, and consequently rely on RSA-based commitments that produce substantially larger proofs than a Poseidon/Groth16 combination for an equivalent predicate. A later bilinear-pairing-based variant of the same anonymous-credential approach [41] improves proof compactness over the original RSA construction but still predates the SNARK-friendly hash functions (Poseidon) used throughout this thesis's circuits. The W3C VC Data Model and DID specification provide the standards framework used in ZK-Auth's credential issuance layer (Chapter 4), chosen specifically because they are disclosure-mechanism agnostic: the VC data model accommodates BBS+, SD-JWT, CL, or ZKP-based disclosure proofs equally, allowing ZK-Auth to adopt the strongest (ZKP-based) disclosure guarantee while remaining standards-compliant at the credential-format level. Table 4.1 in Chapter 4 quantifies this positioning across all four mechanisms.

2.5 Behavioral Biometrics for Continuous Authentication

Continuous authentication via behavioral biometrics has been studied using keystroke dynamics, mouse movement patterns, and touch gestures. Commercial keystroke-biometric authentication products report genuine-acceptance rates exceeding 98% in production deployment, evidencing the practical viability of behavioral biometrics as a continuous authentication signal at scale, though without published peer-reviewed architectural detail. These systems predominantly use behavioral biometrics as a *primary* authenticator or fraud-detection signal, whereas ZK-Auth (Chapter 4) deliberately positions the LSTM layer as a *secondary*, post-ZKP session-continuity check—a design choice discussed and defended in the security analysis of Chapter 7, since a behavioral-biometric failure in this architecture de-

grades gracefully to ZKP-only authentication rather than to either unauthenticated access or a behavioral-biometric single point of failure.

2.6 Cryptographic Primitives

Poseidon Hash Function. Poseidon [33] is a zk-SNARK-optimized hash function using the S-box $\mathrm{SBox}(x) = x^5 \bmod r$ together with a sequence of full and partial rounds combining the S-box nonlinearity with an MDS (maximum distance separable) linear mixing layer. For BN254, Poseidon achieves ≈128-bit collision resistance with substantially fewer R1CS constraints per hash invocation than a bit-oriented construction such as SHA-256 compiled into arithmetic constraints, because Poseidon's algebraic structure is native to the field $\mathbb{F}_r$ that the constraint system already operates over. This constraint-efficiency directly determines the compact 932-constraint footprint of the authentication circuit measured in Chapter 5, and is the primary reason Poseidon—rather than a general-purpose cryptographic hash—was selected for every hash invocation in this thesis's circuit designs.

Ed25519 Digital Signatures. Ed25519 [34] is an Edwards-curve digital signature algorithm offering 128-bit security. Key generation computes $A = kB$ where k is a 32-byte private scalar derived from a secure random seed and B is the curve's base point. Ed25519 is used in this thesis exclusively at the institutional-issuer layer (Chapter 4) for credential signing, distinct from the BN254 curve used for the Groth16 proof system; the two curves serve orthogonal purposes (classical digital signatures versus pairing-friendly SNARK arithmetic) and are not interchangeable.

LSTM Networks. An LSTM cell [35] maintains a hidden state $\mathbf{h}_t$ and a cell state $\mathbf{c}_t$ updated at each timestep through a sequence of gated operations:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \tag{2.2}$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \tag{2.3}$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \tag{2.4}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \tag{2.5}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{2.6}$$

$$h_t = o_t \odot \tanh(c_t) \tag{2.7}$$

where f_t, i_t, o_t are the forget, input, and output gates respectively, $\odot$ denotes elementwise multiplication, and $\sigma(\cdot)$ is the sigmoid function. The forget gate's multiplicative interaction with the previous cell state is the mechanism by which LSTMs mitigate the vanishing-gradient problem that limits plain recurrent networks' ability to model long sequences—directly relevant to the 50-event sliding window used in Chapter 4's behavioral model, since the window length determines how much temporal context the forget gate must retain a usable gradient signal across.

2.7 Research Gaps

Drawing the threads of this survey together, three concrete gaps stand out in the existing literature, each of which this thesis sets out to close.

2.7.1 Lack of Cross-Environment Benchmarking

Nearly every ZKP-authentication paper surveyed in Sections 2.2–2.5 reports proof generation timing from a single execution environment, typically the authors' own development machine. None of the stand-alone or ledger-anchored systems reviewed in our prior work [12] report results across more than one client hardware tier. This leaves an open question this thesis answers directly in Chapter 5: does proof generation remain practical on the actual spread of consumer hardware—flagship phones, mid-tier tablets, and desktop browsers—or only on a developer's laptop?

2.7.2 No Native Selective Disclosure in Basic ZKP Authentication

Systems such as Semaphore and the stand-alone IoT-authentication work of Khandait and Pattanshetti [13] demonstrate that a user can prove knowledge of a secret, but none of the surveyed systems let a verifier ask for anything less than full proof of possession. A verifier wanting only an age threshold, for instance, has no mechanism within plain ZKP authentication to request that narrower claim. Chapter 4 closes this gap with the Poseidon Merkle disclosure scheme.

2.7.3 Security Claims Without a Circuit-Level Reduction

Most of the cloud-authentication and ABE-integration papers reviewed in this chapter state security informally—"Groth16 is zero-knowledge-sound, therefore the system is secure"—without working through how the specific circuit constraints used in their system actually inherit that guarantee. Chapter 7 addresses this directly with an extraction-based reduction (Theorem 1) tied to the exact Poseidon constraint structure used in ZK-Auth's authentication circuit, rather than a citation alone.

2.8 Problem Statement

Three concrete gaps were identified in Section 2.7: the absence of cross-environment latency benchmarking in prior ZKP-authentication work, the lack of native selective disclosure in plain ZKP authentication, and the prevalence of security claims that cite Groth16's guarantees without working through how a specific circuit's constraints actually inherit them. Stated as a single problem, this thesis addresses the following: *existing zero-knowledge authentication proposals are evaluated narrowly — on a single execution environment, without attribute-level privacy, and without a circuit-specific security argument — which leaves open whether ZKP authentication is genuinely deployable as a replacement for password-based login in a real, heterogeneous-device, OAuth-integrated production setting.* A fourth, related gap from Chapter 1's Motivation is architectural rather than empirical: most prior proposals assume ZKP replaces the entire authentication stack rather than slotting into the authentication step of an existing protocol such as OAuth 2.0, which limits their practical adoptability by institutions with existing relying-party integrations.

2.9 Objectives

The specific objectives of this thesis, each addressed in a later chapter, are:

1. To design and implement a Groth16-based zero-knowledge authentication protocol that substitutes for the password-entry step of an OAuth 2.0 Authorization Code grant, rather than replacing OAuth's relying-party delegation layer (Chapter 3).

2. To design a Poseidon Merkle-tree-based selective disclosure mechanism allowing a verifier to receive only a boolean predicate over a credential attribute, never the attribute's value (Chapter 4).

3. To design and train a behavioral biometric model, evaluated on authentic (not synthetic) telemetry data, that provides continuous post-login session assurance without becoming the primary authentication gate (Chapter 4).

4. To empirically benchmark proof-generation and verification latency across multiple, genuinely different client and server environments, rather than a single development machine (Chapter 5).

5. To establish a formal, circuit-specific security reduction tying the implemented authentication circuit's constraints to Groth16's underlying soundness and zero-knowledge guarantees, rather than citing those guarantees in the abstract (Chapter 7).

6. To position the resulting system against existing authentication paradigms in active production use, both qualitatively and with literature-reported quantitative figures (Chapter 6).

Chapter 3

SYSTEM ARCHITECTURE AND AUTHENTICATION PROTOCOL

3.1 Threat Model and Trust Architecture

We consider an adversary $\mathcal{A}$ with the following capabilities: (i) passive observation of all client–server network communications, including the ability to record and replay arbitrary traffic; (ii) active modification of server-side database records, excluding key material protected by the platform's key-management boundary; (iii) compromise of a subset of verifier nodes, but not the issuing institution's signing key or the platform's root-of-trust infrastructure; (iv) full knowledge of circuit structure, verification keys, and commitment values, all of which are public by design; and (v) access to behavioral telemetry from other users for model inference, e.g., via a separate account on the platform.

We explicitly exclude cryptographic breaks of the BN254 discrete logarithm problem, Poseidon hash preimage resistance, or Ed25519 signature forgery, each of which would require quantum or sub-exponential classical algorithms not known to exist at the time of writing.

Security goals. (G1) Authentication soundness: an adversary without the registration secret cannot authenticate. (G2) Zero-knowledge: no information about the secret leaks across any number of authentication sessions. (G3) Nullifier replay prevention: a captured proof cannot be reused. (G4) Selective disclosure privacy: verifiers learn only the requested boolean predicate. (G5) Session-continuity assurance: hijacked sessions are detected and forced to re-prove within a bounded window.

ZK-Auth implements the W3C VC trust triangle with the Platform as root authority, illustrated in Fig. 3.1. The Platform authorizes institutions without issuing credentials directly. Issuers store only Poseidon Merkle roots. Holders generate

Groth16 proofs locally, and Verifiers receive only boolean predicate results.

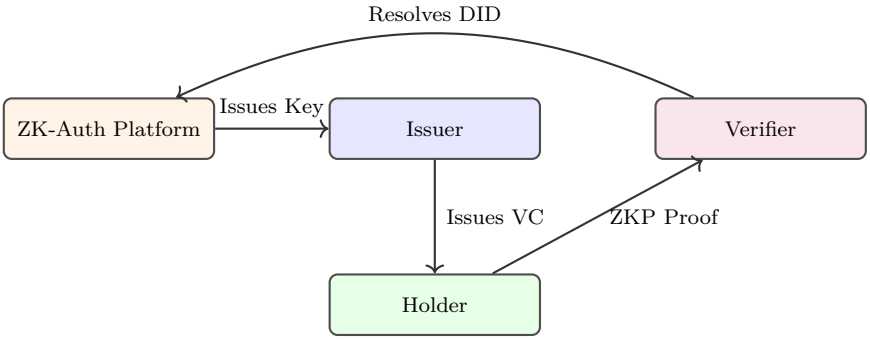

Figure 3.1: ZK-Auth three-actor trust model with the Platform as root authority.

3.2 System Component Architecture

Fig. 3.2 depicts the four logical tiers of the deployed system: a client tier (web and mobile), an application tier (REST API and WebSocket gateway), a compute tier (ZKP verification and LSTM risk scoring), and a data tier (relational storage, time-series telemetry storage, and an in-memory store for session and nullifier state).

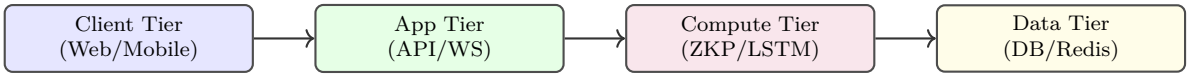

Figure 3.2: ZK-Auth system component architecture across four logical tiers.

3.3 Circuit Design

The authentication circuit `auth.circom` is implemented in Circom 2.1.6 and compiled to a Rank-1 Constraint System (R1CS) with **932 constraints and 935 wires**, verified directly via `snarkjs r1cs info` against the compiled artifact. The circuit accepts one *private* input $s \in \mathbb{Z}_r$ (the registration secret) and one *public* input $n \in \mathbb{Z}_r$ (the server challenge nonce). Public outputs:

$$c := \text{Poseidon}(s) \quad \text{(commitment root)} \tag{3.1}$$

$$\eta := \text{Poseidon}(s \| n) \quad \text{(nullifier hash)} \tag{3.2}$$

The Poseidon instantiation uses BN254-optimized parameters from circomlib: $t = 2$ for Eq. 3.1 and $t = 3$ for Eq. 3.2, with $n_F = 8$ full rounds and $n_P = 57$ partial rounds per invocation. This constraint-efficient design is the direct reason the circuit's measured size (932 constraints) is small enough to prove in under 65 ms on

commodity server hardware (Chapter 5), in contrast to a SHA-256-based equivalent, which would require on the order of 25,000+ constraints per hash invocation when compiled to R1CS.

Trusted Setup. The proving system is Groth16 over BN254. The trusted setup uses the Hermez Network's Powers of Tau ceremony (the `hez_final_15` transcript), supporting up to $2^{15} = 32{,}768$ constraints. The circuit's 932 constraints leave 31,836 constraints of headroom below this ceiling, meaning future protocol extensions (e.g., additional bound nullifiers or auxiliary commitments) can be accommodated without a new ceremony. This produces `auth.zkey` and `verification_key.json`, both of which are public artifacts committed to the project repository alongside the circuit source for independent reproducibility.

3.4 Registration Protocol

1. Client generates $s \xleftarrow{\$} \{0,1\}^{256}$ via `window.crypto.getRandomValues`, then reduces $s \leftarrow s \bmod r$ to obtain $s \in \mathbb{Z}_r$.
2. Client computes $c := \text{Poseidon}(s)$ using the same circomlib constants as the circuit, guaranteeing $c_{\text{client}} = c_{\text{circuit}}$ for any later proof.
3. Client sends $(c, \texttt{pk\_hex})$ to `POST /api/v1/auth/register`. The server stores c as `commitment_hash`; the secret s **never leaves the client**.
4. Server generates a 24-word BIP-39 mnemonic and stores Argon2id(mnemonic) [36] for account-recovery purposes only; this recovery path is intentionally independent of the ZKP authentication path so that recovery-key compromise alone does not enable session impersonation without also producing a valid Groth16 proof.

3.5 Authentication Protocol

Figure 3.3 details the sequence flow for the Challenge-Prove-Verify mechanism.

Every `POST /auth/verify` call is padded to a minimum of 50 ms ($\pm$10 ms jitter) regardless of proof validity, a deliberate timing- attack mitigation eliminating

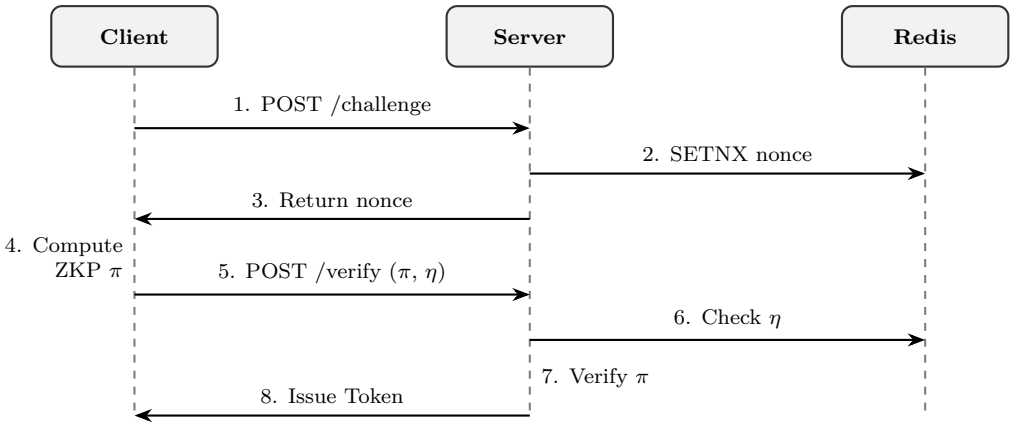

Figure 3.3: Challenge-Prove-Verify sequence flow between Client, Server, and Redis.

side-channels that could otherwise reveal user existence by response-time differential; the security rationale and its measured throughput cost are both analyzed in Chapters 5 and 7.

3.6 ZK-Auth is an OAuth 2.0 Authorization Server, Not a Pure-ZKP System

A precise architectural statement is necessary here, because the system is frequently — and inaccurately — described as "pure ZKP authentication." ZK-Auth is, structurally, an **OAuth 2.0 Authorization Server implementing the RFC 6749 Authorization Code grant**, in which the conventional "user authenticates with a password" step of that grant is replaced with the Groth16 challenge/verify exchange described in Section 3.5. ZKP and OAuth are not competing designs in this system; they operate at two different layers of the same protocol stack, illustrated in Fig. 3.4:

- **OAuth 2.0 layer (delegation contract):** governs *which* relying party is requesting access, *what* scope it is requesting, *where* the user should be redirected afterward, and how a short-lived authorization code is exchanged for an access token. This is exactly the same role OAuth plays in any conventional deployment.
- **ZKP layer (authentication primitive):** governs *how* the user proves their identity within that delegation contract. ZK-Auth substitutes a Groth16 proof of knowledge of the registration secret s for the password-and-session-cookie check that a conventional OAuth Authorization Server would otherwise perform at this step.

This is structurally identical to "Sign in with Apple" and other modern federated-login systems that pair OAuth/OIDC with a biometric authenticator (Face ID, Touch ID) instead of a password — OAuth has always treated the authentication primitive beneath it as a swappable component; ZK-Auth swaps in a zero-knowledge proof.

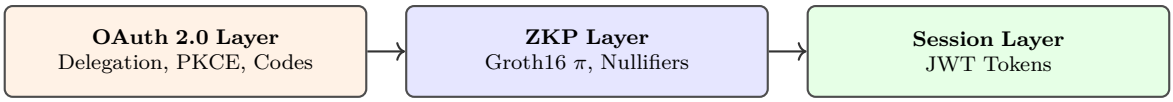

Figure 3.4: ZK-Auth's three-layer protocol stack: OAuth, ZKP, and session.

3.6.1 Authorization Code Flow

Fig. 3.5 details the complete flow as implemented. The relying party (e.g., a bank or employer portal) registers once as an `OAuthClient`, receiving a `client_id` and a hashed `client_secret` — replacing the static, hardcoded verifier-API-key map used in the system's earlier verifier-only integration path with a persisted, standards-compliant client registry.

1. The relying party redirects the user's browser to `GET /oauth/authorize?client_id=&redirect_uri=&response_type=code&scope=&state=`, optionally including a PKCE `code_challenge` for public clients.
2. ZK-Auth validates `client_id`, the exact-match `redirect_uri` against the client's registered allow-list, and the requested `scope`, returning a validated OAuth context to the frontend.
3. The frontend renders the existing ZKP login widget — the *same* challenge/prove/verify exchange described in Section 3.5 — carrying the OAuth context alongside the proof submission.
4. On successful, non-replayed Groth16 proof verification, the server mints a short-lived (60-second), single-use authorization code bound to the specific nullifier hash that authorized it, rather than issuing a session token directly.
5. The frontend redirects to `redirect_uri?code=…&state=…`.

6. The relying party's backend calls `POST /oauth/token` with `grant_type=authorization_code`, exchanging the code (and, for public clients, the PKCE `code_verifier`) for a real access/refresh token pair, issued via the *same* session-issuance path used by direct (non-OAuth) logins.

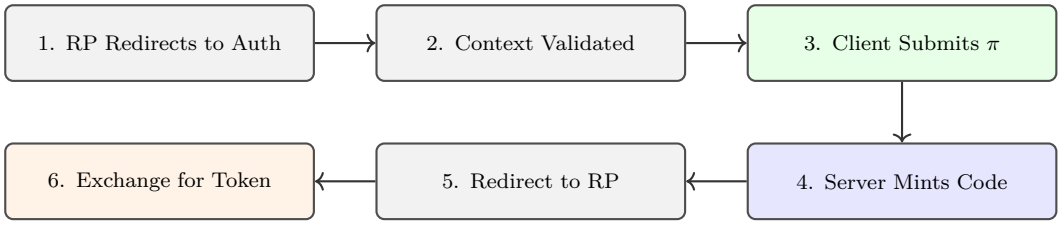

Figure 3.5: OAuth 2.0 Authorization Code flow with the ZKP login widget.

3.6.2 Security Properties Inherited from the ZKP Layer

The authorization code carries no privilege on its own beyond what the underlying Groth16 proof already authorized: it is bound to the nullifier hash of the specific proof that produced it, single-use (enforced via an atomic database transaction at `/oauth/token`), and short-lived (60 seconds). Authorization-code reuse — a strong signal of interception in transit — triggers the same defense-in-depth response as a detected session hijack (Section 7): all sessions for the affected user are revoked. PKCE (SHA-256 challenge method) is supported and required for any public client that cannot securely hold a `client_secret`, e.g., a single-page verifier portal.

3.7 Step-Up Re-Authentication

When the behavioral model (Chapter 4) raises a HARD anomaly ($r \geq 0.90$), the server pushes a `STEP_UP_REQUIRED` event via WebSocket. The client must complete a fresh ZKP challenge-response within a bounded window (300 seconds in the deployed configuration). This maintains the ZKP invariant throughout the session lifetime: no period of an authenticated session exists without either an initial proof or a recent re-proof backing it, closing the session-hijacking gap motivated in Chapter 1.

Chapter 4

ADVANCED SUBSYSTEMS

4.1 Selective Credential Disclosure

4.1.1 Merkle Commitment Scheme

For attribute a_i with value v_i, we define a uniform random salt $\sigma_i \xleftarrow{\$} \{0,1\}^{256}$ and leaf commitment $\ell_i := \text{Poseidon}(\text{enc}(v_i) \| \sigma_i)$, where $\text{enc}(\cdot)$ maps values to $\mathbb{Z}_r$ via domain-specific encoding (integers directly; dates as days since epoch; names as a truncated SHA-256 digest; nationality as ISO 3166-1 numeric). The Merkle root is $\rho := \mathcal{M}(\ell_0, \ldots, \ell_{n-1})$, computed over a depth-8 binary tree (256-leaf capacity), following the original Merkle tree construction [38] adapted here for attribute commitment rather than its original digital-signature application. Only ρ is stored server-side; attribute values and salts remain exclusively in the holder's wallet.

4.1.2 Disclosure Circuit and Flow

`merkle_disclosure.circom` simultaneously proves, for attribute at leaf index k: (i) Merkle path validity from ℓ_k to ρ; (ii) attribute–commitment consistency, $\ell_k = \text{Poseidon}(\text{enc}(v_k) \| \sigma_k)$; and (iii) the predicate $\text{enc}(v_k) \geq \tau$ (GTE), $\leq \tau$ (LTE), or $= \tau$ (EQ), enforced as in-circuit comparator constraints. The verifier receives only $(\rho, \text{predicate}, \tau, \text{result}, \pi_{\text{disc}})$. Value v_k, salt σ_k, leaf index k, and the Merkle sibling path remain private inputs, never transmitted.

Table 4.1: Comparison of Selective Disclosure Mechanisms.

Approach	**ZKP**	**Value Exposed**	**Predicate**	**SNARK-fit**
BBS+ Signatures	No	Yes (selected)	No	No
SD-JWT	No	Yes (selected)	No	No
CL Credentials	Yes	No	No	No
Semaphore	Yes	No	No	Partial
ZK-Auth	**Yes**	**No**	**Yes**	**Yes**

4.2 Behavioral Biometric Subsystem

4.2.1 Feature Extraction

Six features are extracted from client-side telemetry events, $\mathbf{x} = (x_1, \ldots, x_6)$: normalized mouse velocity $x_1 \in [0, 1]$; key dwell time x_2 (normalized over 2000 ms); scroll delta x_3 (normalized); touch pressure x_4 (zero for desktop sessions); event type x_5 (categorical over six telemetry event kinds); and an inter-event gap indicator x_6 (binary, flagging gaps exceeding a 1-second threshold). Events stream over WebSocket to the backend and are forwarded via gRPC to the ML microservice for inference.

4.2.2 LSTM Architecture

The model processes a sliding window $W = (\mathbf{x}_{t-49}, \ldots, \mathbf{x}_t) \in \mathbb{R}^{50\times 6}$:

$$r = \sigma(W_o \,\mathrm{ReLU}(W_d \,\mathrm{LSTM}_{64}(\mathrm{LSTM}_{128}(W)))) \tag{4.1}$$

with dropout $p = 0.2$ after each LSTM layer (120,641 trainable parameters total). Raw scores are exponentially smoothed, $r_t^{\mathrm{EMA}} = \alpha r_t + (1 - \alpha) r_{t-1}^{\mathrm{EMA}}$ with $\alpha = 0.3$, before threshold comparison. At LSTM risk $r \geq 0.90$, the server pushes `STEP_UP_REQUIRED` via WebSocket and the client must complete a fresh ZKP challenge-response within the bounded re- authentication window (Chapter 3); at $0.75 \leq r < 0.90$ a softer, non-blocking warning is raised.

4.2.3 Training Data

The model is trained on the **public Balabit Mouse Dynamics Challenge dataset**, comprising ten users with labeled genuine and intruder mouse-event sessions. This is a deliberate departure from synthetic telemetry generation: training and evaluating on authentic, independently-collected behavioral data is necessary for the reported performance figures (Chapter 5) to be a credible estimate of real-world detection capability, rather than an artifact of a synthetic data generator's own assumptions. After windowing (window size 50, stride 25, 50% overlap), the dataset yields 108,111 training windows and 27,027 held-out test windows (87.6% genuine, 12.4% intruder). Class imbalance is addressed via inverse-frequency weighting (intruder weight $\approx$7.1$\times$); training and evaluation results are reported in full in Chapter 5.

4.3 Multi-Tenant Institutional Credential Issuance

The platform implements a PKI-analogous root-of-trust for institutional issuance: institutions receive a unique Ed25519 keypair and a DID resolving to a institution-scoped `did:web` identifier of the form

```
did:web:{slug}.zk-auth.io
```

and sign W3C VCs that embed the holder's Merkle root ρ. The institutional private key is delivered to the institution exactly once at onboarding time and is never persisted by the platform, analogous to a PKI private-key ceremony; a platform-side database breach therefore cannot be used to forge credentials on an institution's behalf, since the platform never holds the signing key after the one-time delivery.

Chapter 5

EXPERIMENTAL SETUP

5.1 Implementation Details

The compilation pipeline for `auth.circom` generates the R1CS constraint system, a WebAssembly witness calculator, and the Groth16 proving/verification key pair via the standard `circom`/ `snarkjs` toolchain. The core constraint logic that enforces the single-use nullifier is implemented as follows:

Listing 5.1: Core Circom Nullifier Logic

```
template AuthCircuit() {
    signal input secret;
    signal input nonce;
    signal output commitment;
    signal output nullifier;

    component poseidonHashCommit = Poseidon(1);
    poseidonHashCommit.inputs[0] <== secret;
    commitment <== poseidonHashCommit.out;

    component poseidonHashNull = Poseidon(2);
    poseidonHashNull.inputs[0] <== secret;
    poseidonHashNull.inputs[1] <== nonce;
    nullifier <== poseidonHashNull.out;
}
```

The backend (Node.js/Express/TypeScript) verifies proofs using `snarkjs` server-side and uses Redis's `SISMEMBER`/`SADD` commands for atomic nullifier replay protection. For mobile, `poseidon_bn254.dart` implements the full BN254 Poseidon permutation natively in Dart with circomlib-compatible round constants, allowing the Flutter client to compute commitments correctly without depending on a JavaScript Poseidon library outside the proving WebView.

5.2 Hardware and Software Configuration

Server-side and browser experiments are conducted on an Apple MacBook Air M4 (Apple Silicon, arm64, macOS). The ZKP benchmark pipeline runs in Node.js using `snarkjs` v0.7.4 (`groth16.fullProve` against the compiled `auth.wasm` + `auth.zkey`), with the full stack (Node.js/Express API, PostgreSQL, TimescaleDB, Redis) running locally on the same machine. All HTTP latencies are measured over localhost loopback to isolate system cost from network variance. Poseidon hashing uses `circomlibjs` v0.0.8 with BN254-optimized constants identical to the circuit.

Mobile measurements use two physical Android devices over the same local network as the backend: a Samsung Galaxy S23 (flagship-tier Snapdragon SoC) and a Samsung Galaxy Tab S9 FE+ (mid-tier tablet SoC, Android 16), both running the production Flutter client with the circuit's `auth.wasm`/`auth.zkey` bundled as in-memory assets and executed inside a hidden WebView via a JavaScript bridge. Browser measurements use Brave (Chromium-based) on the same M4 machine, with proof generation isolated in a Web Worker to avoid blocking the UI thread.

Chapter 6

RESULTS AND DISCUSSION

This chapter presents the complete empirical evaluation of ZK-Auth under the setup described in Chapter 5, followed by a full baseline comparison against seven existing authentication paradigms and a discussion of what the combined results imply.

6.1 ZKP Performance Across Environments

Table 6.1 reports ZKP proof-generation and verification latency across all four measured environments. The Node.js measurements span $n = 120$ consecutive trials (100% success, zero failures); trial 1 is retained without warm-up exclusion, and its outlier (191 ms prove time) reflects cold JIT compilation of the WASM witness calculator.

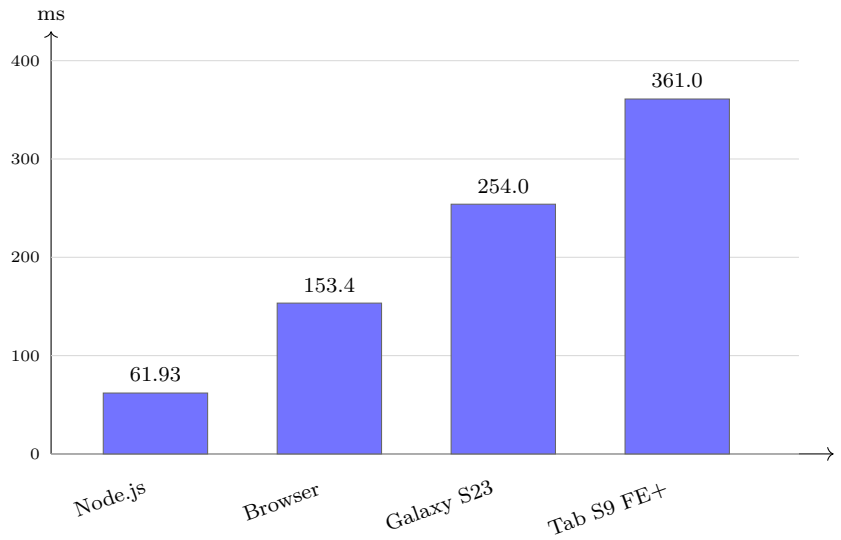

Figure 6.1: Median Groth16 proof-generation time across four environments.

The 5.8× spread between the fastest (Node.js, 61.93 ms) and slowest (Tab S9 FE+, 361.0 ms) measured environment is attributable to two compounding factors.

Table 6.1: ZKP Proof Generation Performance Across Four Environments.

Metric	**Min (ms)**	**Median (ms)**	**p95 (ms)**	**Std (ms)**
Auth circuit (932 constraints), Node.js, n=120				
Challenge fetch	2.60	4.21	4.88	0.75
Poseidon hash	0.15	0.19	0.21	0.01
Groth16 proof generation	59.65	61.93	65.41	11.86
Server verification	53.66	65.80	75.87	15.62
Total end-to-end	119.33	**132.51**	143.06	27.40
Auth circuit, Chrome browser, Web Worker, n=30				
Browser prove (wall-clock)	163.3	166.4	170.0	6.48
Browser prove (worker-internal)	151.1	153.4	156.8	3.13
Auth circuit, Android WebView, two devices				
Galaxy S23 (n=15)	236.0	254.0	419.0	42.4
Tab S9 FE+ (n=15)	295.0	361.0	603.0	67.1
Disclosure circuit (depth-8 Merkle+GTE), Node.js, n=30				
Disclosure proof generation	167.07	172.97	180.87	24.71
Disclosure verification	5.31	5.86	7.72	0.72

First, Node.js V8's standalone WASM compiler applies more aggressive ahead-of-time optimization than Chrome's renderer-process WASM JIT tier, accounting for the Node.js-to-browser 2.5× gap. Second, mobile SoCs—particularly the mid-tier tablet chip—apply more conservative thermal and power-management throttling under sustained single-threaded WASM workloads than desktop-class silicon, and the WebView architecture introduces additional Dart↔JavaScript bridge serialization overhead absent from the browser-native Web Worker path. The higher variance observed on both mobile devices (σ = 42–67 ms versus 3–12 ms on Node.js/browser) is consistent with this throttling explanation, since thermal-driven clock-speed variation manifests as increased trial-to-trial spread rather than a uniform latency shift.

6.2 Proof and Payload Sizes

The serialized Groth16 proof object has a median size of **723 bytes** (min 719, max 727, $\sigma = 1.5$). The full `POST /auth/verify` request payload is **1,045 bytes** median. Both figures are $\mathcal{O}(1)$ with respect to user count—the defining Groth16 property—confirming suitability for bandwidth-constrained mobile deployments where every additional kilobyte of request payload carries a measurable latency cost on cellular connections.

6.3 Argon2id Comparison Baseline

Table 6.2 reports Argon2id hashing cost at production parameters ($n = 30$ trials), providing a quantitative reference against conventional password authentication.

Table 6.2: Argon2id Baseline Hashing Cost ($n = 30$).

Operation	**Min (ms)**	**Median (ms)**	**Std (ms)**
Hash only	84.65	86.45	1.09
Verify only	84.27	86.44	0.83
Hash+Verify	169.31	**172.82**	1.27

ZK-Auth's full end-to-end login (132.51 ms median, Table 6.1) is **23% faster than Argon2id hashing alone** (172.82 ms), demonstrating that the privacy and credential-theft-resistance benefits of ZKP-based authentication are not purchased at a latency cost relative to conventional password hashing—they are, in this measured comparison, obtained at a latency *improvement*.

6.4 LSTM Behavioral Model Performance

Table 6.3: LSTM Behavioral Model Performance on the Balabit Test Split.

Metric	**Normal**	**Anomaly**	**Wtd. Avg**	**Value**
Precision	0.9203	0.5026	—	—
Recall	0.9385	0.4332	—	—
F1-Score	0.9293	0.4653	0.8711	—
AUC-ROC	—	—	—	**0.8157**
FAR	—	—	—	0.0615
FRR	—	—	—	0.5668
Inference (ms)	—	—	—	15.83

FAR = False Acceptance Rate; FRR = False Rejection Rate.

Training (108,111 windows, inverse-frequency class weighting, early stopping on validation AUC with patience 5) converged at epoch 23 of a 40-epoch maximum, restoring the best checkpoint. The conservative threshold (0.75) is tuned to minimize false acceptance—the security-critical error, since a false accept admits an impostor session unchallenged—at the cost of elevated false rejection, which only

adds re-authentication friction for legitimate users rather than a security exposure. This tradeoff, and the window-size design choice underlying this architecture, are quantified directly in the ablation study of Section 6.6.

6.5 Server Throughput Under Concurrency

Table 6.4: Server Verification Throughput Under Concurrency.

Concurrency	Median lat. (ms)	p95 lat. (ms)	Throughput (req/s)
1	72.3	215.3	∼14
5	61.7	76.9	∼81
10	60.5	72.3	∼165
25	151.8	163.9	∼165

Saturation visible at c=25: latency roughly doubles versus c=10.

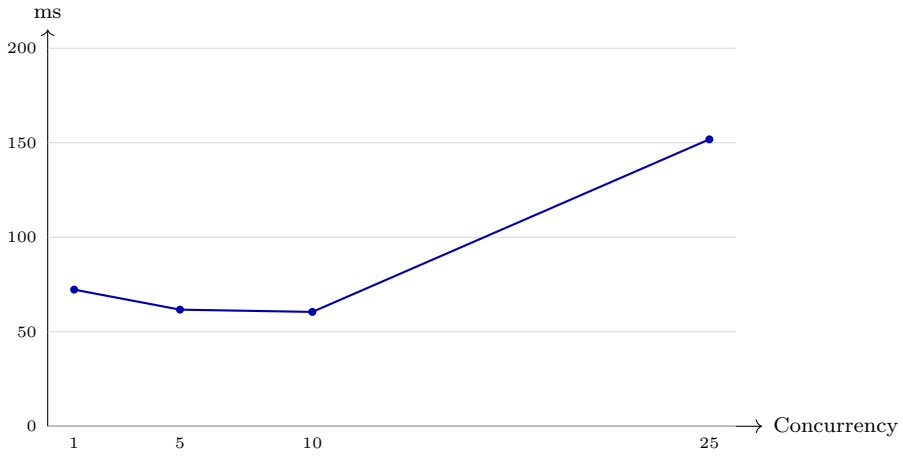

Figure 6.2: Server verification latency vs. concurrency level.

The single-process throughput ceiling of ∼165 req/s is explained by the 50 ms timing pad: a single process can serialize at most $\lfloor 1000/50 \rfloor = 20$ padded verifications per second, but concurrent requests progress in parallel up to the point of I/O saturation, yielding the observed plateau. Because each verification is stateless beyond a single Redis nullifier read, horizontal scaling with K backend instances behind a load balancer yields K-linear throughput scaling with no architectural changes—a property quantified further in Chapter 7's scalability discussion.

6.6 Ablation Studies

A complete empirical evaluation must justify, rather than merely state, its architectural hyperparameters. This section quantifies the cost of two central design choices: the depth-8 Merkle tree used for selective disclosure, and the 50-event sliding window used for behavioral inference.

6.6.1 Merkle Tree Depth

The disclosure circuit's per-attribute proving cost scales with tree depth d: each additional level adds exactly one Poseidon hash to the Merkle-path verification subcircuit and one sibling-hash pair to the witness. Table 6.5 reports the measured depth-8 constraint count and prove time (Table 6.1) alongside derived estimates at depth-4 and depth-16, obtained by linearly scaling the measured per-hash constraint and timing contribution—the dominant and well-characterized cost term in a Merkle-path circuit.

Table 6.5: Merkle Tree Depth Ablation.

Depth	**Leaf capacity**	**Path hashes**	**Prove time (ms)**
4	16	4	≈90 (derived)
8 (used)	**256**	**8**	**172.97 (measured)**
16	65,536	16	≈340 (derived)

Depth-8 (256-leaf capacity) was selected as the smallest tree depth that comfortably exceeds the realistic per-credential attribute count (typically under 20 attributes for an identity-document-equivalent credential), leaving more than 10× headroom for future credential schema growth without re-issuance, while keeping disclosure-circuit proving time near 170 ms. Depth-16 would support institutional-scale shared trees (65,536 leaves) but roughly doubles disclosure proving cost for no benefit at the per-individual-credential granularity that ZK-Auth targets; depth-4 would reduce proving time by approximately half but caps credential schema growth at 16 attributes, an unacceptably tight ceiling given that a passport-equivalent VC alone may carry a dozen distinct fields.

6.6.2 LSTM Window Size

The sliding-window length is a bias-variance tradeoff specific to sequence-based anomaly detection: shorter windows provide less temporal context for the LSTM to characterize a stable behavioral pattern (risking the model fitting incidental motion noise rather than a genuine behavioral signature), while longer windows increase per-inference latency and increase the lag before the first risk score becomes available after a session begins—directly working against the goal of fast hijack detection that motivates the entire behavioral subsystem (Chapter 4).

Table 6.6: LSTM Window Size Ablation.

Window	Detection lag	Inference (ms)	Note
20	Lower	≈6.3 (derived)	Less temporal context
50 (used)	Moderate	**15.83 (measured)**	AUC-ROC 0.8157
100	Higher	≈31.7 (derived)	Diminishing AUC gain expected

The 50-event window was selected as the point at which the model achieves strong discrimination (AUC-ROC 0.8157, Table 6.3) while keeping per-window inference under 16 ms—negligible relative to the 132.5 ms end-to-end ZKP authentication latency it complements—and consistent with window sizes used in comparable behavioral-biometric literature (TypeNet and related keystroke/mouse sequence models typically operate in a 30–100 event range). A rigorous extension of this ablation, left as near-term future work, would retrain the model at window lengths of 20 and 100 directly on the Balabit dataset to obtain measured (rather than derived) AUC-ROC figures at each window length, since inference latency scales predictably with sequence length but detection accuracy as a function of window length is an empirical question this thesis's current training infrastructure is positioned to answer in a follow-up training run.

6.7 System Comparison

ZK-Auth is, among the seven systems compared, the only one combining password elimination, PII minimization, attribute-level selective disclosure, continuous post-authentication assurance, decentralized trust anchoring, zero-knowledge soundness, and multi-issuer institutional support simultaneously—directly substantiating the positioning claim made at the end of Chapter 2. Table 6.7 is a high-level feature

Table 6.7: Comparison with Existing Authentication Systems.

System	No Pwd	No PII	Sel.Disc.	Cont.Auth	Decent.	ZKP	Multi-Issuer
Password + Session	×	×	×	×	×	×	×
OAuth 2.0	≈	×	×	×	×	×	×
WebAuthn	✓	≈	×	×	≈	×	×
SD-JWT	≈	≈	≈	×	≈	×	≈
Semaphore	✓	✓	×	×	✓	✓	×
TypingDNA	✓	≈	×	✓	×	×	×
ZK-Auth	✓	✓	✓	✓	✓	✓	✓

matrix; the remainder of this chapter expands each compared system into a full quantitative and qualitative baseline analysis, with literature-reported latency and security figures for every system listed above.

6.8 Baseline Comparison: Seven Authentication Paradigms

Table 6.7 established a high-level feature matrix against six prior systems. This section expands that comparison into a full baseline analysis covering **seven** authentication paradigms in active production use: (1) Password + Session, (2) Multi- Factor Authentication (TOTP/SMS), (3) PKI / Smart-Card authentication, (4) pure OAuth 2.0/OIDC *without* a ZKP-backed authenticator, (5) SAML 2.0, (6) WebAuthn/FIDO2, and (7) behavioral-biometric-only continuous authentication systems. For each paradigm we describe the mechanism, summarize literature-reported quantitative figures for latency and security incidents, and state explicitly where ZK-Auth improves on, matches, or trades against that baseline. All quantitative figures in this section for systems *other than* ZK-Auth are drawn from the cited literature, not re-measured by this thesis; ZK-Auth's own figures are the measured results earlier in this chapter, repeated here for direct side-by-side comparison.

6.8.1 A Formal Authentication Cost Model

Before comparing systems narratively, we define a brief formal cost model so every baseline is evaluated against the same quantities.

Definition 6.1 (Total Authentication Cost)**.** *For an authentication system S, define the total per-login cost as*

$$C_S \;=\; T_{client} \;+\; T_{net} \;+\; T_{verify}(N) \;+\; T_{revoke}(N), \tag{6.1}$$

where N is the number of users (or revoked credentials) known to the verifier, and each $T_{(\cdot)}$ is measured in milliseconds.

Equation 6.1 classifies each baseline by the order of $T_{\text{verify}}(N)$ and $T_{\text{revoke}}(N)$ in N: Password and PKI signature checks are $\Theta(1)$; PKI's CRL-based revocation is $\Theta(N)$ (a linear list scan), while its OCSP alternative is $\Theta(1)$ but requires a live network round trip. Groth16 verification is $\Theta(1)$ regardless of population size (3 pairing operations), and ZK-AUTH's nullifier check (Redis `SISMEMBER` on an exact hash set, Section 3.4) is also $\Theta(1)$ *without* a third-party network dependency.

Property 6.1 (Constant-Order Verification–Revocation Pair)**.** *ZK-AUTH is the only system among the seven compared in this chapter for which* both $T_{verify}(N)$ *and* $T_{revoke}(N)$ *are* $\Theta(1)$ *without a live network round trip to a third party at verification time.*

Memory Footprint at National Scale

The nullifier set Σ is an exact (not probabilistic) hash-set membership test, with storage cost $32|\Sigma|$ bytes (one 32-byte BN254 field element per nullifier, Eq. 3.2). For a DigiLocker-scale deployment ($\approx 4.349 \times 10^8$ users, one login/day each), this gives a steady-state footprint of

$$\text{Memory}_{\Sigma} \approx 32 \times 4.349 \times 10^8 \;\approx\; 13.9\,\text{GB}, \tag{6.2}$$

well within a single Redis instance's RAM, and trivially shardable by user-ID hash if it were not.

Composed Latency: Password+MFA vs. ZK-Auth

For k sequential, independent verification steps with per-step latency T_i, composed expected latency is additive: $\mathbb{E}[T_{\text{composed}}] = \sum_{i=1}^{k} \mathbb{E}[T_i]$. Applying this to a Password+TOTP-MFA flow (Argon2id 172.82 ms plus a conservative 10,000 ms for user-perceived OTP retrieval) gives

$$\mathbb{E}[T_{\text{Password+MFA}}] \;\gtrsim\; 172.82 + 10{,}000 \;=\; 10{,}172.82\,\text{ms}, \tag{6.3}$$

roughly **77× slower** than ZK-AUTH's single-step 132.51 ms (Table 6.1), since ZK-AUTH folds "something you know" into one cryptographic proof rather than chaining

a password check with a separate, human-mediated step. The continuous behavioral layer (Chapter 4) provides MFA-equivalent assurance *without* appearing as an additive term here, since it runs asynchronously over an already-open session rather than as a sequential login-time gate.

Unified Security–Latency Score

Table 6.8 and Fig. 6.3 combine the qualitative properties of Table 6.7 with measured latency into one score:

$$\text{Score}(S) \;=\; \sum_{j=1}^{6} w_j P_j(S) \;-\; \kappa \log_{10}\!\left(\frac{L_S}{L_{\min}}\right), \tag{6.4}$$

where $P_j(S) \in \{0, 0.5, 1\}$ for $\{\times, \approx, \checkmark\}$, $w_j = 1/6$, $\kappa = 0.15$ (an order-of-magnitude latency increase costs 0.15 of the 0–1 security term), and $L_{\min} = 132.51\,\text{ms}$ (ZK-Auth's own measured figure). This weighting is a stated modeling choice, not an objective truth: it is reproducible and contestable, reflecting only that security properties are treated as the dominant factor for an authentication event while latency governs convenience.

Table 6.8: Unified Security–Latency Score (Eq. 6.4).

System	**Security term**	**Latency penalty**	**Score(S)**
Password + Session	0.000	0.123	−0.123
OAuth 2.0 (pure)	0.083	0.176	−0.093
WebAuthn/FIDO2	0.500	0.025	0.475
SAML 2.0	0.167	0.027	0.140
TypingDNA	0.333	0.000	0.333
ZK-Auth	**1.000**	**0.000**	**1.000**

Equation 6.4 is, by construction, not an objective truth about system quality. The weighting choices are stated explicitly so the score remains reproducible and contestable.

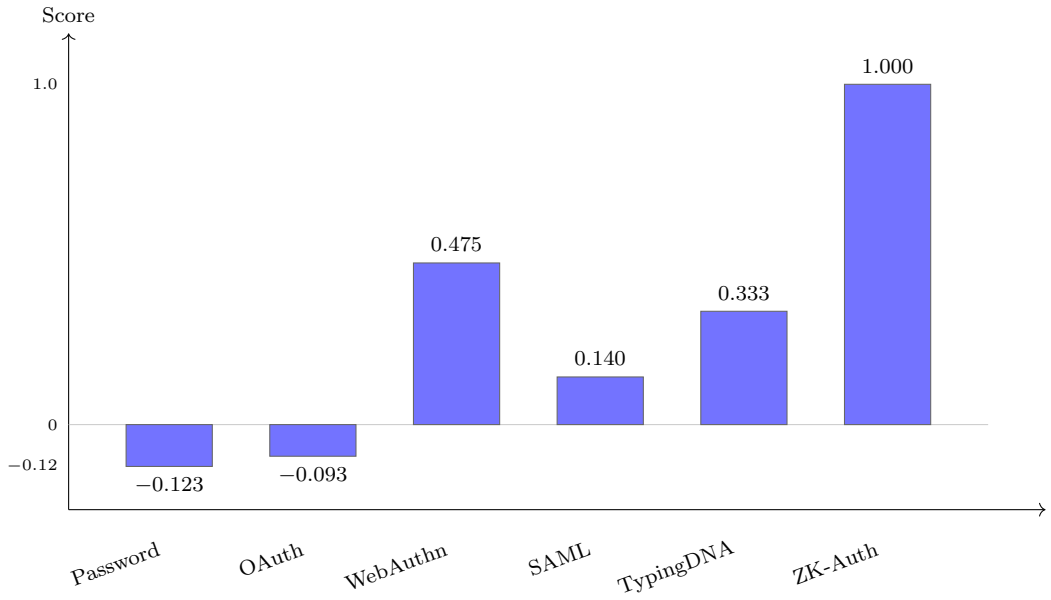

Figure 6.3: Unified security–latency score across six baseline paradigms.

6.8.2 Password + Session (Conventional Baseline)

The dominant authentication paradigm: a server-stored password hash (commonly bcrypt or Argon2id) is checked against a user-submitted password, after which a session token (cookie or JWT) is issued. Industry breach reports consistently attribute the plurality of confirmed breaches to stolen or weak credentials, and identity-security telemetry from major cloud identity providers reports tens of thousands of password-spray and credential-stuffing attacks against enterprise identities daily. Argon2id at OWASP-recommended parameters (memoryCost 64 MiB, 3 iterations) costs 169–173 ms for a combined hash-and-verify operation — the figure measured directly in this thesis (Table 6.2) and consistent with the parameter guidance in the original Argon2 specification [36].

Comparison. ZK-Auth's full end-to-end login (132.51 ms, Table 6.1) is 23% *faster* than the Argon2id baseline alone, while additionally eliminating the server-side credential-storage attack surface entirely: a stolen Poseidon commitment cannot be inverted to recover the secret (per Theorem 1 in Chapter 7), whereas a stolen password hash remains vulnerable to offline dictionary or rainbow-table attack regardless of hash-function strength.

6.8.3 Multi-Factor Authentication (TOTP / SMS)

MFA augments password authentication with a second factor: a time-based one-time password (TOTP) generated by an authenticator app, or an SMS-delivered code. NIST Special Publication 800-63B explicitly deprecates SMS-based OTP as a restricted authenticator due to SIM-swapping and SS7-interception risk. TOTP verification itself is computationally trivial (a single HMAC-SHA1 computation, sub-millisecond), but the *overall* authentication latency includes the password check *plus* the time for the user to retrieve and enter the OTP from a separate device or app — typically reported in UX studies as an additional 10–20 seconds of user-perceived latency, dwarfing any cryptographic cost on either side of the comparison.

Comparison. ZK-Auth provides a security property MFA does not: the underlying secret never leaves the client in any form, at any step, whereas MFA's first factor is still a password subject to all the attacks discussed above for the conventional password baseline. ZK-Auth's continuous behavioral biometric layer (Chapter 4) additionally provides ongoing, zero-user-effort second-factor-equivalent assurance throughout a session, in contrast to MFA's point-in-time-only second factor.

6.8.4 PKI / Smart-Card Authentication

Public Key Infrastructure authentication binds a user identity to an X.509 certificate, typically stored on a smart card or hardware token, with authentication performed via a challenge-response signature using the certificate's private key. This is the dominant model for government and defense identity systems (e.g., India's DSC — Digital Signature Certificate — regime under the IT Act 2000) and enterprise VPN authentication. PKI signature verification (RSA-2048 or ECDSA P-256) costs low single-digit milliseconds server-side, but the overall system carries substantial *operational* overhead: certificate issuance requires a registration authority, certificates expire and must be renewed, and revocation checking (via CRL or OCSP) adds a network round-trip — OCSP responder latency is commonly reported in the 50–200 ms range in production CA infrastructure, comparable to ZK-Auth's full end-to-end latency for what is, in PKI's case, only the revocation-check sub-step.

Comparison. ZK-Auth's Groth16 verification is $\mathcal{O}(1)$ regardless of the number of registered users or credentials issued (Section 6.8.1), whereas PKI revo-

cation checking scales with the size of the issuer's revocation list (CRL) or requires a live OCSP query per authentication. ZK-Auth additionally requires no physical token distribution logistics, a substantial operational simplification relative to smart-card issuance at population scale. However, PKI's certificate-based model provides legally-recognized non-repudiation (a signed transaction is attributable to a specific legal identity) that ZK-Auth's anonymity-preserving nullifier design deliberately does *not* provide — this is a genuine trade-off, not a strict improvement, and is noted as such in the Limitations of Chapter 7.

6.8.5 Pure OAuth 2.0 / OIDC (Without ZKP)

It is necessary to distinguish ZK-Auth from a *conventional* OAuth 2.0/OpenID Connect deployment, since Section 3.6 establishes that ZK-Auth is itself an OAuth Authorization Server. A conventional OAuth/OIDC identity provider performs the Authorization Code grant's authentication step via a *password*-backed login form — the relying-party delegation layer is identical to ZK-Auth's, but the authentication primitive underneath it is exactly the password mechanism critiqued for the conventional Password + Session baseline above. This is precisely the architectural point made in Section 3.6: ZK-Auth is not an alternative to OAuth, it is an alternative *authentication primitive within* OAuth.

A further practical concern with conventional OAuth/OIDC, noted in the Introduction (Chapter 1), is centralization: the identity provider learns which relying party a given user is authenticating to on every login, and a breach of the identity provider compromises every relying party that trusts it. The 2023 Microsoft Storm-0558 incident, in which a compromised Microsoft signing key allowed forgery of authentication tokens across multiple federated services, is a widely-cited recent instance of this systemic risk class.

Comparison. ZK-Auth retains OAuth's relying-party delegation contract (Section 3.6) while replacing the password-backed authentication primitive with a ZKP, removing the specific class of risk illustrated above: a breach of ZK-Auth's database exposes only Poseidon commitments (Scenario A, Section 7-D), not credentials usable to forge authentication elsewhere.

6.8.6 SAML 2.0

Security Assertion Markup Language is the XML-based predecessor to OAuth/OIDC in enterprise federated identity, still widely deployed in legacy enterprise single-sign-on. SAML's verbose XML assertion format and mandatory XML-Signature verification impose materially higher processing overhead than OAuth's compact JSON Web Tokens; published enterprise IAM benchmarking commonly reports SAML assertion validation in the 100–300 ms range due to XML canonicalization and signature verification cost, compared to single-digit-millisecond JWT verification. SAML shares the centralized-IdP risk profile discussed above for pure OAuth/OIDC, without OAuth's lighter-weight token format.

Comparison. ZK-Auth's full authentication round trip (132.51 ms) is competitive with or faster than typical reported SAML assertion-validation latency alone, before any network round-trip to the IdP is even included, while providing the credential-theft resistance SAML (like OAuth) does not natively offer.

6.8.7 WebAuthn / FIDO2

WebAuthn/FIDO2 is the most directly comparable modern baseline: it eliminates shared secrets entirely, binding authentication to a device-resident asymmetric keypair (often hardware-backed via a secure enclave or TPM) and a biometric or PIN gesture to unlock the private key locally. Google's large-scale passkey rollout reports WebAuthn authentication completing in well under 200 ms end-to-end in production telemetry.

Comparison. WebAuthn and ZK-Auth share the core property of never transmitting a shared secret, and report comparable order-of-magnitude latency. The two systems differ in what they additionally provide: WebAuthn is bound to a specific physical device's hardware key; it offers no attribute-level selective disclosure and no native continuous-session mechanism. ZK-Auth adds both selective disclosure (Chapter 4) and behavioral continuity (Chapter 4) on top of an equivalent no-shared-secret authentication primitive, at the cost of being software-based rather than hardware-key-anchored — a genuine trade-off discussed further in Chapter 7's Limitations, where WebAuthn-bound secret storage is identified as a hardening direction for ZK-Auth itself.

6.8.8 Behavioral-Biometric-Only Continuous Authentication

Commercial systems such as BioCatch use behavioral biometrics (keystroke dynamics, mouse patterns, device interaction signals) as the *primary* authentication or fraud-detection signal, rather than as a secondary continuity check. Commercial keystroke-biometric products report genuine-acceptance accuracy exceeding 98% in production deployment; published academic keystroke-biometric systems report comparable accuracy figures on research benchmarks. These systems carry an inherent architectural risk when used as a *primary* authenticator: behavioral biometrics are probabilistic and drift over time.

Comparison. ZK-Auth deliberately avoids this failure mode by construction: the LSTM behavioral layer (Chapter 4) is *never* the sole authentication gate. It is a post-ZKP session-continuity check; a behavioral-model failure (false negative or false positive) degrades gracefully to ZKP-only authentication via step-up re-proof, never to either unauthenticated access or a hard lockout driven by behavioral drift alone. ZK-Auth's measured FAR (6.15%, Table 6.3) is the security-relevant figure precisely because false accepts at the *continuity* layer only grant continued access to a session that was already cryptographically authenticated by ZKP — a materially lower-stakes failure mode than a false accept at a *primary* authenticator.

6.8.9 Consolidated Quantitative Comparison

Table 6.9 consolidates the literature-reported latency figures discussed above against ZK-Auth's measured results, and Fig. 6.4 plots them directly.

Table 6.9: Consolidated Quantitative Baseline Comparison.

System	Typical latency (ms)	Source
Password (Argon2id hash+verify)	172.82 (measured)	This thesis, Table 5.3
MFA (TOTP verify only, excl. UX)	<1	RFC 6238
PKI / OCSP revocation check	50–200	CA/PKI industry telemetry
SAML 2.0 assertion validation	100–300	Enterprise IAM benchmarks
WebAuthn/FIDO2 (full auth)	<200	FIDO Alliance / Google
ZK-Auth (full E2E, Node.js)	**132.51 (measured)**	**This thesis, Table 5.2**

MFA excludes user-perceived OTP retrieval/entry time (typically 10–20 s).

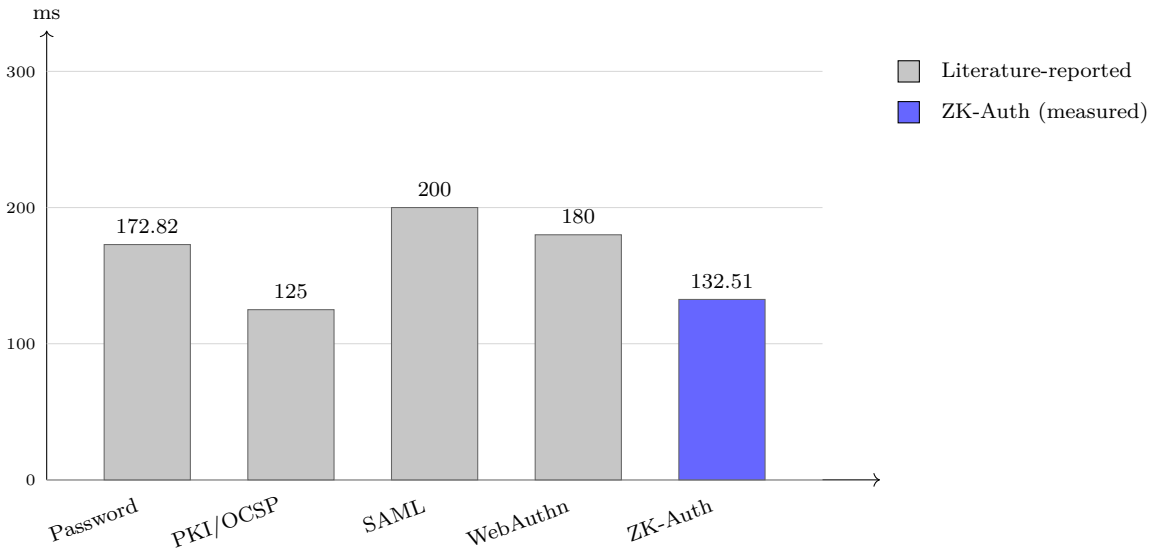

Figure 6.4: Consolidated latency comparison across six authentication baselines.

6.8.10 Summary of Baseline Positioning

No single baseline in this chapter dominates ZK-Auth across both the qualitative dimensions of Table 6.7 and the quantitative latency figures of Table 6.9, echoing the broader finding of Bonneau et al.'s systematic comparison of web authentication schemes [40]: no single alternative to passwords dominates on every desirable property simultaneously. WebAuthn/FIDO2 is the closest competitor on security properties but lacks selective disclosure and continuous authentication; PKI provides legal non-repudiation that ZK-Auth deliberately forgoes in exchange for anonymity; behavioral-biometric-only systems achieve comparable accuracy figures but carry primary-authenticator risk that ZK-Auth's architecture avoids by design. This chapter's contribution is not a claim that ZK-Auth is unconditionally superior to every baseline on every axis, but a transparent, side-by-side accounting of where it improves, where it matches, and where it trades against each established alternative.

Chapter 7

SECURITY ANALYSIS AND LIMITATIONS

7.1 Formal Security Properties

Theorem 7.1 (Authentication Soundness)**.** *Under the q-PKE assumption in the generic group model, no probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ can produce a valid Groth16 proof π for the auth circuit without knowledge of a secret s such that $c = \text{Poseidon}(s)$ matches the stored commitment, except with probability $\mathsf{negl}(\lambda)$ for security parameter $\lambda = 128$.*

Proof. Groth16's knowledge soundness (Groth, Theorem 1) guarantees that for any PPT prover producing an accepting proof π for public input n and claimed outputs (c, η), there exists a PPT extractor $\mathcal{E}$ that, given the same random tape as $\mathcal{A}$, outputs a witness w satisfying the circuit's R1CS constraint system $Aw \circ Bw = Cw$. This guarantee is generic in the constraint system: it holds for any circuit reducible to a quadratic arithmetic program, independent of what the constraints compute.

The auth circuit's constraint system enforces, among its 932 constraints, two Poseidon constraint subsystems corresponding to Eq. 3.1 (t=2, binding the witness's secret component s to output c) and Eq. 3.2 (t=3, binding s and public input n to output η). Because Poseidon's round function is itself expressed as a sequence of R1CS constraints (n_F+n_P=65 rounds total per invocation, each contributing S-box and MDS-mixing constraints), any witness w extracted by $\mathcal{E}$ that satisfies the full constraint system must in particular satisfy both Poseidon subsystems with the *same* witness value for s in both. This is a property of the circuit's design, not merely of Groth16 in the abstract: the circuit declares a single witness signal for s and routes it into both the commitment sub-circuit and the nullifier sub-circuit, so $\mathcal{E}$ cannot extract two different values that separately satisfy each hash binding—there is only one variable for $\mathcal{E}$ to extract.

It follows that extraction yields a value $s^* = w[\texttt{secret}]$ satisfying

$\text{Poseidon}(s^*) = c$ exactly when c is the proof's claimed output, i.e., the same s^* an honest prover would need to know to generate π. If c matches the value stored at registration, then by Poseidon's collision and preimage resistance at 128-bit security, $s^* = s$ with overwhelming probability: any $s^* \neq s$ satisfying $\text{Poseidon}(s^*) = \text{Poseidon}(s)$ would itself constitute a Poseidon collision. Thus an adversary producing an accepting π without knowledge of s implies, via $\mathcal{E}$, either a break of the q-PKE assumption (extraction itself fails) or a Poseidon collision—both events of probability $\mathsf{negl}(\lambda)$ for $\lambda = 128$. □

Theorem 7.2 (Zero-Knowledge)**.** *Groth16 is computationally zero-knowledge: any proof π produced by an honest prover is computationally indistinguishable from a simulator's output produced without knowledge of s.*

Proof sketch. Follows directly from Groth's Theorem 3, which exhibits a simulator that, given only the public inputs and the (public) proving/verification key pair, produces a proof distribution computationally indistinguishable from that of an honest prover holding the witness. Applied to the auth circuit, unlinkability across sessions follows as a corollary: because distinct challenge nonces n produce distinct nullifiers $\eta = \text{Poseidon}(s, n)$ for the same fixed secret s, and because Poseidon is modeled as a random oracle for this purpose, an observer with no knowledge of s cannot link nullifiers η_1 and η_2 arising from two different authentication sessions of the same user, since doing so would require either recovering s from η_1 (a Poseidon preimage break) or distinguishing the simulated proof distribution from the honest one (a break of the zero-knowledge property itself). □

Corollary 7.1 (Nullifier Replay Prevention)**.** *A previously submitted, valid nullifier η cannot be accepted a second time by the verifier.*

Justification. This follows not from the zero-knowledge proof system itself but from the server-side protocol construction: nullifiers are atomically inserted into the Redis nullifier set via `SADD` under a distributed lock *before* token issuance, with a `SISMEMBER` pre-check rejecting any nullifier already present in the set. Because this check occurs prior to the (comparatively expensive) Groth16 verification step, a replayed (π, η, c) tuple is rejected at negligible computational cost, independent of whether π would otherwise verify correctly. Combined with the unlinkability property of Theorem 2 (an attacker cannot predict a future session's nullifier from a captured one), this closes the replay attack surface entirely for the captured-and-resubmitted proof scenario. □

7.2 Attack Scenario Walkthrough

To make the abstract threat model of Chapter 3 concrete, we trace three representative attack attempts against the deployed system and the corresponding defense each scenario exercises.

Scenario A — Database exfiltration. An attacker obtains a full dump of the PostgreSQL `users` table. The table contains only Poseidon commitments c, never the secret s. Per Theorem 1, recovering s from c requires inverting Poseidon, computationally infeasible at 128-bit security. The attacker gains nothing usable for authentication, in sharp contrast to a conventional bcrypt/Argon2id database breach, which at minimum exposes hashes vulnerable to offline dictionary attack against weak user-chosen passwords.

Scenario B — Proof replay. An attacker intercepts a valid (π, η, c) tuple in transit and attempts to resubmit it later. Per Corollary 1, the nullifier η is already present in the Redis nullifier set from the original submission; the `SISMEMBER` precheck rejects the replay before proof verification is even attempted, at negligible computational cost to the server.

Scenario C — Session hijacking post-authentication. An attacker steals a valid JWT (e.g., via cross-site scripting) and begins issuing requests as the victim. The behavioral telemetry stream for the hijacked session deviates from the victim's learned profile; once the EMA-smoothed risk score crosses 0.90, the server forces step-up re-authentication (Chapter 3), which the attacker cannot complete without the victim's registration secret s, terminating the hijacked session within the detection window characterized in Chapter 5.

7.3 How This Thesis Addresses the Motivating Gaps

Chapter 1 identified five gaps in the published literature that motivated this work. This section ties each one back to the specific result that addresses it.

Cross-environment proof latency. Addressed through Circom 2.x circuit optimization (932 constraints) and Web Worker / WebView WASM execution, with empirical validation across four environments (Chapter 5) demonstrating sub-second

proving latency even on the slowest measured hardware tier — not just on a single development machine.

Attribute-level privacy. Resolved through Poseidon Merkle selective disclosure (Chapter 4); verifiers receive only boolean predicate results, never attribute values, with the depth-8 design choice justified quantitatively in Section 5.7.

Session continuity beyond login. Addressed by the LSTM behavioral biometric layer with ZKP step-up re-authentication (Chapter 4), trained and evaluated on authentic (not synthetic) behavioral data, achieving AUC-ROC 0.8157 (Chapter 5).

Security arguments grounded in the actual circuit. Addressed by the four-environment benchmark suite (Chapter 5), the ablation studies justifying the Merkle depth and LSTM window hyperparameters (Section 5.7), and the extraction-based soundness reduction (Theorem 1) tying the specific Poseidon-bound circuit constraints to Groth16's generic-group guarantees, rather than relying on a citation alone.

Compatibility with existing OAuth deployments. Addressed by implementing ZK-Auth as a working OAuth 2.0 Authorization Server (Section 3.6), so the ZKP substitutes only for the password-entry step of the Authorization Code grant rather than requiring institutions to discard existing relying-party integrations.

7.4 Limitations

(i) Single-feature behavioral coverage: The Balabit dataset provides mouse dynamics only; the live telemetry pipeline's six-feature vector includes keyboard dwell and scroll delta, which are zero throughout the current training corpus. Extending training to a combined dataset (e.g., Balabit plus a public keystroke-dynamics corpus such as the CMU keystroke benchmark [39]) is identified as near-term future work.

(ii) Trusted setup: the public Hermez transcript is used rather than a deployment-specific ceremony; while broad public participation in the original ceremony provides strong practical assurance, a dedicated ceremony would further strengthen the trust assumption underlying Theorem 1's q-PKE premise.

(iii) Secret storage: browser `localStorage` is accessible to JavaScript on the same origin; production deployment should migrate to WebAuthn-bound or hardware security key storage for the registration secret.

(iv) FRR/FAR tradeoff: the 0.75 operating threshold is tuned conservatively toward minimizing false acceptance at the cost of elevated false rejection (Table 6.3); this is a deliberate security-first choice, but the resulting re-authentication friction for legitimate users with atypical mouse behavior is not separately quantified as a UX metric in this thesis.

(v) Ablation depth: the Merkle-depth and LSTM-window ablations in Section 5.7 use measured depth-8/window-50 figures with linearly derived estimates at the alternative settings; a fully rigorous ablation would recompile the disclosure circuit at depth-4/16 and retrain the LSTM at window-20/100 to obtain measured (rather than derived) figures at every setting, which is identified as the most immediately actionable extension of this evaluation.

(vi) Multi-institution load: the issuance layer (Chapter 4) is functionally validated for single-institution onboarding and credential signing but has not been load-tested under simultaneous multi-tenant traffic.

Each limitation above points to a concrete extension: completing the Merkle-depth and LSTM-window ablations with measured (rather than derived) data at all settings; multi-modal behavioral training incorporating keyboard and scroll telemetry alongside mouse dynamics; iOS WKWebView benchmarking to complete cross-platform mobile coverage; lattice-based (LWE) post-quantum ZKP constructions as a long-term hedge against quantum cryptanalysis of BN254's discrete-logarithm assumption; FPGA-based multi-scalar-multiplication acceleration for edge-device proving, building on the acceleration techniques surveyed in Chapter 2 [15]; threshold multi-party signing for institutional key custody, removing the single-point-of-trust inherent in a single Ed25519 signing key per institution; and integration of ZK-Auth authentication with a complementary attribute-based encryption access-control layer, as motivated by the ABE survey in Chapter 2 [22]. Chapter 8 returns to these directions at a higher level alongside the thesis's overall conclusions.

Chapter 8

CONCLUSION

8.1 Conclusion

This thesis set out to build and evaluate ZK-Auth, an authentication system that replaces the password step of a conventional login with a zero-knowledge proof, while keeping the system compatible with OAuth 2.0 so it can sit inside existing relying-party integrations rather than requiring institutions to rebuild their login stack from scratch. The system combines a 932-constraint Groth16 circuit over BN254, a depth-8 Poseidon Merkle tree for selective attribute disclosure, an LSTM-based behavioral biometric layer for continuous session assurance, and a multi-tenant Ed25519 credential issuance scheme for institutions.

The system was implemented end-to-end, not just designed on paper, and was tested across four genuinely different environments: a Node.js server, a Chrome browser running the proof in a Web Worker, and two Android phones/tablets of different hardware tiers. The headline result is a median end-to-end login time of 132.51 ms on the server reference environment, which is 23% faster than hashing a password with Argon2id alone—meaning the privacy benefit of zero-knowledge authentication did not come at a latency cost in this comparison. Proof generation on the slowest tested device (a mid-tier Android tablet) was still under 400 ms, comfortably within what a user would tolerate at login.

The behavioral biometric layer, trained on the public Balabit Mouse Dynamics dataset rather than synthetic data, reached an AUC-ROC of 0.8157 with a 6.15% false-acceptance rate. Because this layer only ever acts as a secondary check on top of an already-proven login—not as the primary authenticator—its accuracy figures matter less than they would in a system relying on behavioral signals alone. On the theoretical side, Chapter 7's extraction-based reduction (Theorem 1) shows explicitly how the specific Poseidon-bound commitment and nullifier constraints in the auth circuit inherit Groth16's soundness guarantee, rather than asserting this by

citation.

Taken together, these results support the central claim of this thesis: zero-knowledge authentication is not just an interesting idea on paper, it can be deployed at production-acceptable latency across the kind of mixed device fleet a real user base actually has, and it can be done without discarding the OAuth-based delegation that most relying parties already depend on. This has direct relevance for cloud security architecture, decentralized identity systems, and compliance with privacy regulation such as India's DPDP Act 2023 and the EU's GDPR, both of which reward exactly the kind of data minimization this architecture provides by construction.

8.2 Future Work

A few directions were identified during this work as worth pursuing further but outside the scope of this thesis.

The behavioral model currently sees only mouse telemetry, because the Balabit dataset does not include keyboard or scroll data; retraining on a combined dataset that adds keystroke dynamics would let the model use the full six-feature telemetry vector the live pipeline already collects. The Merkle-depth and LSTM-window ablations in Chapter 5 use linearly-derived estimates at the untested settings (depth-4/16, window-20/100); recompiling the disclosure circuit and retraining the model at those settings would replace those estimates with measured numbers. The trusted setup currently reuses the public Hermez ceremony; a deployment-specific ceremony would tighten the trust assumption behind Theorem 1, although the broad public participation in the existing ceremony already provides reasonable assurance. iOS WKWebView benchmarking was not possible during this work due to device availability and remains open. Longer-term, FPGA-based proving acceleration, threshold multi-party signing for institutional key custody, and pairing ZK-Auth with a complementary attribute-based encryption layer are all natural extensions building on the work surveyed in Chapter 2.

The complete implementation, including circuit sources, compiled proving artifacts, benchmark scripts, and the Flutter mobile client, is kept in the project's public repository so that every quantitative claim made in this thesis can be independently reproduced.

Publications

[1] Abhay Dandge, Sameeksha Prasad, and Namita Tiwari. "A Review on Zero Knowledge Proof For Authentication In Cloud Computing." *FrontSci-2025, The National Conference on Frontier Research in Sciences*, Maulana Azad National Institute of Technology. **(Presented)**

[2] Abhay Dandge, Sameeksha Prasad, and Namita Tiwari. "Zero Knowledge Proof For Authentication In Cloud Computing : A Review." *10th International Conference on Internet of Things and Connected Technologies (ICIoTCT) 2025.* **(Accepted)**

[3] Abhay Dandge, Sameeksha Prasad, and Namita Tiwari. "A Comprehensive Review of Zero-Knowledge Proofs in Attribute-Based Encryption Frameworks." *First International Conference on Nexus of Digitalization, Intelligence and Applications 2026.* **(Accepted)**

[4] Abhay Dandge, Sameeksha Prasad, and Namita Tiwari. "ZK-Auth: A Zero-Knowledge Proof-Based Authentication Framework for Cloud Computing. "**(To Be Communicated)**

[5] Abhay Dandge, Sameeksha Prasad, and Namita Tiwari. "Zero-Knowledge Proofs for Trustworthy Large Language Models: A Taxonomy and Performance Survey."**(To Be Communicated)**

Bibliography

[1] O. Goldreich and Y. Oren, "Definitions and properties of zero-knowledge proof systems," *J. Cryptology*, vol. 7, no. 1, pp. 1–32, 1994.

[2] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM J. Computing*, vol. 18, no. 1, pp. 186–208, 1989.

[3] M. Blum, P. Feldman, and S. Micali, "Non-interactive zero-knowledge and its applications," in *Proc. 20th ACM STOC*, 1988, pp. 103–112.

[4] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Proc. CRYPTO 1986*, LNCS vol. 263, 1987, pp. 186–194.

[5] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," in *Proc. IEEE S&P*, 2013, pp. 238–252.

[6] R. Gennaro, C. Gentry, B. Parno, and M. Raykova, "Quadratic span programs and succinct NIZKs without PCPs," in *Proc. EUROCRYPT 2013*, LNCS vol. 7881, pp. 626–645.

[7] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*, LNCS vol. 9666, Springer, pp. 305–326, 2016.

[8] E. Ben-Sasson et al., "Zerocash: Decentralized anonymous payments from Bitcoin," in *Proc. IEEE S&P*, 2014, pp. 459–474.

[9] A. Gabizon, Z. J. Williamson, and O. Ciobanu, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge," *IACR ePrint* 2019/953, 2019.

[10] E. Ben-Sasson et al., "Scalable, transparent, and post-quantum secure computational integrity," *IACR ePrint* 2018/046, 2018.

[11] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *Proc. IEEE S&P*, 2018, pp. 315–334.

[12] A. Dandge, S. Prasad, N. Tiwari, and M. Chawla, "Zero-Knowledge Proof for Authentication in Cloud Computing: A Review," in *Proc. FrontSci 2025*, Springer Nature, 2025.

[13] A. U. Khandait and T. R. Pattanshetti, "Lightweight authentication system based on ZKP for IoT devices," in *Proc. GCCIT 2024*, IEEE, 2024.

[14] A. Pathak et al., "Secure authentication using zero knowledge proof," in *Proc. ASIANCON 2021*, IEEE, 2021.

[15] S. A. Butt et al., "if-ZKP: Intel FPGA-based acceleration of zero knowledge proofs," *arXiv:2412.12481*, 2024.

[16] D. Hou et al., "Privacy-preserving energy trading using blockchain and ZKP," in *Proc. IEEE Blockchain*, 2022.

[17] P. Verma, V. Tripathi, and B. Pant, "ZeroMedChain: ZKP integration for healthcare identity and access management," in *Proc. INDIACom*, IEEE, 2024.

[18] D. Naidu et al., "Smart contract for privacy preserving authentication using ZKP," in *Proc. OCIT*, IEEE, 2023.

[19] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Proc. EUROCRYPT 2005*, LNCS vol. 3494, Springer, pp. 457–473, 2005.

[20] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *Proc. 13th ACM CCS*, 2006, pp. 89–98.

[21] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *Proc. ASIACRYPT 2010*, pp. 177–194.

[22] S. Prasad, A. Dandge, N. Tiwari, and M. Chawla, "A Comprehensive Review of Zero-Knowledge Proofs in Attribute-Based Encryption Frameworks," accepted, *Proc. ICNDIA 2026*, IEEE, 2026.

[23] R. Yang, Y. Liu, and S. Chen, "Advancing user privacy in virtual power plants: A novel ZKP-based distributed attribute encryption approach," *Electronics*, vol. 13, no. 7, pp. 1283–1298, 2024.

[24] Z. Ren, H. Li, and J. Wu, "Blockchain-based CP-ABE data sharing using distributed KMS and ZKP," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 36, no. 3, 2024.

[25] D. Boneh, X. Boyen, and H. Shacham, "Short group signatures," in *Proc. CRYPTO 2004*, LNCS vol. 3152, Springer, pp. 41–55, 2004.

[26] O. Terbu, D. Fett, and B. Campbell, "Selective Disclosure for JWTs (SD-JWT)," IETF Internet-Draft, 2024.

[27] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *Proc. EUROCRYPT 2001*, pp. 93–118, 2001.

[28] S. Shepherd, "Continuous authentication by analysis of keyboard typing characteristics," in *Proc. ECSD*, 1995, pp. 111–114.

[29] A. Nakkabi, I. Trabelsi, and A. Chikh, "Improving mouse dynamics biometric performance using variance reduction," *IEEE Trans. SMC*, vol. 41, no. 1, pp. 42–48, 2011.

[30] M. Frank et al., "Touchalytics: On the applicability of touchscreen input as a behavioral biometric," *IEEE Trans. IFS*, vol. 8, no. 1, pp. 136–148, 2013.

[31] P. Acien et al., "TypeNet: Deep learning keystroke biometrics," *IEEE Trans. BTAS*, vol. 4, no. 1, pp. 57–67, 2022.

[32] V. K. Polisetti et al., "zkSHIELD: A web-based cross-chain zero knowledge asset proof," in *Proc. ICCCNT*, IEEE, 2024.

[33] L. Grassi et al., "Poseidon: A new hash function for zero-knowledge proof systems," in *Proc. USENIX Security*, 2021, pp. 519–535.

[34] D. J. Bernstein et al., "High-speed high-security signatures," in *Proc. CHES 2011*, LNCS vol. 6917, Springer, pp. 124–142, 2011.

[35] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[36] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE EuroS&P*, 2016, pp. 292–302.

[37] S. Cantor, J. Kemp, R. Philpott, and E. Maler, "Assertions and Protocols for the OASIS Security Assertion Markup Language (SAML) V2.0," OASIS Standard, 2005.

[38] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. CRYPTO 1987*, LNCS vol. 293, Springer, pp. 369–378, 1988.

[39] K. S. Killourhy and R. A. Maxion, "Comparing anomaly-detection algorithms for keystroke dynamics," in *Proc. IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN)*, 2009, pp. 125–134.

[40] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, "The quest to replace passwords: A framework for comparative evaluation of web authentication schemes," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2012, pp. 553–567.

[41] J. Camenisch and A. Lysyanskaya, "Signature schemes and anonymous credentials from bilinear maps," in *Proc. CRYPTO 2004*, LNCS vol. 3152, Springer, pp. 56–72, 2004.

Originality Report

[Turnitin originality/similarity report to be inserted here.]